CE725 CONSTRUCTION ECONOMICS AND FINANCE

Topic 7 — Earned Value Management Dashboard

Project Charter

Team 40

Sajjad Nakhwa	23B0702	Project Lead and Finance Modeller
Dependar Arya	23B0331	Data Lead
Neel Shah	23B4244	AI and Tool Developer
Akshat Karkar	23B4229	Validation Engineer
Pearsh Gumasta	23B4232	Documentation and Reflection Lead

Semester course project · November 2026

> This is the final version of the charter submitted earlier in the semester. The fields below expand on that submission; nothing in it has been reversed. Two points of wording are worth stating plainly, since the submitted charter is the document on record:
>
> - The submitted charter describes an "**AI-assisted**" dashboard. AI assistance was applied to building the tool, and is logged in full in `docs/05-ai-usage-log.md`. The delivered tool calls no language model at runtime; the plain-language commentary is rule-based and deterministic, for the reasons set out under *Scope boundaries* below.
> - The submitted charter lists PV, EV, AC, CPI, SPI, EAC, ETC and TCPI. All are implemented. The tool additionally reports CV, SV, Earned Schedule — SV(t), SPI(t) and IEAC(t) — and all four standard EAC methods rather than one.

Item	Details
Team number	40
Team members and roles	**Sajjad Nakhwa** (23B0702) — Project Lead and Finance Modeller: owns the EVM method, the choice of forecasting formulas and the index conventions used throughout. **Dependar Arya** (23B0331) — Data Lead: owns the case study project, the work breakdown structure, the time-phased budget and the source list. **Neel Shah** (23B4244) — AI and Tool Developer: owns the Python engine, the Streamlit dashboard and the Excel export; manages the repository. **Akshat Karkar** (23B4229) — Validation Engineer: owns the manual benchmark workbook, the hand calculations, the test suite and the discrepancy log. **Pearsh Gumasta** (23B4232) — Documentation and Reflection Lead: owns the user manual, the AI usage log, the reflection report and the presentation.
Topic selected	Topic 7 — Earned Value Management (EVM) Dashboard
Problem statement	Construction project managers learn that a project is in trouble too late to act. Monthly reporting typically compares money spent against money budgeted, which conflates two different failures: spending more than planned, and doing less work than planned. A project that has spent 28% of its budget may be 28% complete or 20% complete, and the cost report alone cannot distinguish them. Earned Value Management separates the two by introducing a third measure, the budgeted cost of the work actually done, and from it derives early warning indices. The problem this tool addresses is that EVM is computationally tedious at work-package level, is therefore usually either skipped or rolled up to a single project curve that cannot locate a variance, and produces indices that non-specialist decision-makers cannot interpret.
Intended users of the tool	Primary: a project manager or planning engineer preparing a monthly progress report for a contractor or a client. Secondary: a client's project management consultant reviewing contractor claims; a lender's independent engineer assessing whether a project will need more funding than sanctioned. Each of these reads indices rather than computing them, which is why the tool leads with a plain-language verdict rather than a table.
Methods to be applied	Earned Value Management per the standard definitions: the base measures PV, EV and AC; variances CV and SV; the performance indices CPI and SPI; the four standard estimate-at-completion methods; ETC, VAC and TCPI. Extended with **Earned Schedule** (Lipke, 2003), which re-expresses schedule performance in time rather than currency and removes the well-known defect by which SPI returns to 1.00 at completion however late a project finishes. Earned value is measured by the **percent-complete method** at work-package level.

Item	Details
Data sources	A synthetic case study project designed for the purpose: a G+4 RCC academic block, BAC ₹14.00 crore, 12-month baseline, 19 work packages, reported monthly with a data date at the end of Period 6. Budgets are order-of-magnitude plausible for an institutional building of this size built to CPWD DSR-style item rates. **All figures are assumed and are not drawn from any tender, estimate or invoice.** The reasoning behind the choice of synthetic over real data is set out in the method note.
Build platform	Python 3.12 for the calculation engine; Streamlit with Plotly for the dashboard; openpyxl for the Excel export and the manual benchmark workbook; pytest for the test suite; LibreOffice in headless mode to evaluate the benchmark workbook's formulas during validation. All source is in a public Git repository.
Benchmark case for validation	All EVM indices for **reporting periods 5 and 6** of the case study, computed independently in a plain Excel workbook in which every figure is a typed formula acting on the raw inputs. The two implementations — Python and Excel formulas — are compared measure by measure. The Excel side is *evaluated* by a real spreadsheet engine rather than re-implemented in Python, because a Python re-implementation would only confirm our own understanding of the formulas against itself.
Key risks to completion	**1. Misunderstanding a formula consistently.** The most dangerous failure mode: an error made identically in both the tool and the hand check passes validation. Mitigated by writing the hand calculation from the textbook definitions before the engine, and by asserting algebraic identities (TCPI against EAC1 must equal CPI; VAC on EAC2 must equal CV) that hold only if the whole chain is correct. **2. Plausible-looking wrong data.** A case study tuned to produce interesting indices can quietly become internally inconsistent. Mitigated by strict input validation that raises rather than warns, and by reconciliation checks that the time-phased budget sums to each package budget and to BAC. **3. Over-reliance on AI-generated code.** Mitigated by the validation strategy above and by a test suite whose expected values are hand-derived, not generated. **4. Scope creep into schedule analysis.** EVM touches CPM scheduling, and modelling the network would not have been finishable. Explicitly out of scope; the tool consumes a time-phased budget and does not compute a critical path.

Scope boundaries

Stating what the tool does **not** do is as much a part of the charter as what it does, because every omission below is a place where a user could reasonably expect more.

In scope

- Work-package-level EVM, so that a variance can be attributed rather than merely detected.
- All standard indices, variances and the four standard EAC methods.
- Earned Schedule and time-based schedule indices.
- Plain-language monthly commentary generated deterministically.
- Export to a live-formula Excel workbook that recalculates when edited.

Out of scope

- **Critical path and network analysis.** The tool reads a time-phased budget. It does not know which activities are on the critical path, and therefore cannot say whether a given slip delays completion.
- **Re-baselining.** One baseline per project. A project that has been formally re-baselined needs the new baseline loaded as a new project.

- **Resource and cash flow modelling.** No retention, no mobilisation advance, no interest on overdraft. These matter enormously to a contractor and are the subject of a different topic in this brief.
- **Variation and claim management.** Scope changes are assumed to have been incorporated into the budget before the data reaches the tool.
- **Multi-currency and escalation.** All figures are nominal rupees at one price base.

The limitation that governs everything else

Earned value is percent complete multiplied by budget. Percent complete is a judgement made at site by a person with an interest in the answer.

If a work package is reported at 60% complete when it is really 40%, the tool will compute a confident, precise, internally consistent and entirely wrong earned value, and every index derived from it inherits the error. Validation against a manual benchmark proves the *arithmetic* is right. It cannot prove the *measurement* is right, and nothing in software can.

This is stated in the charter, repeated in the user manual, repeated on the dashboard's data tab, and repeated in the exported workbook, because it is the single thing a user most needs to carry away and the single thing a polished interface is most likely to obscure.

CE725 CONSTRUCTION ECONOMICS AND FINANCE

Topic 7 — Earned Value Management Dashboard

Method Note and Data Sources

Team 40

Sajjad Nakhwa	23B0702	Project Lead and Finance Modeller
Dependar Arya	23B0331	Data Lead
Neel Shah	23B4244	AI and Tool Developer
Akshat Karkar	23B4229	Validation Engineer
Pearsh Gumasta	23B4232	Documentation and Reflection Lead

Semester course project · November 2026

1. The method, and why this one

1.1 What EVM is for

A conventional monthly cost report compares two numbers: what was budgeted to have been spent by now, and what has actually been spent. The gap between them is reported as a variance. The trouble is that this single comparison conflates two entirely different problems.

A project that has spent ₹4.00 crore against a budgeted ₹4.02 crore looks healthy. It is healthy if it has also done ₹4.00 crore of work. It is in serious difficulty if it has done ₹3.47 crore of work, because it is then both behind programme and overspending, and the apparent agreement between budget and spend is a coincidence of two errors pointing in opposite directions. That is precisely the position of the case study project used throughout this work.

EVM resolves this by introducing a third measure. Instead of two numbers there are three, all in the same units:

Measure	Definition	The question it answers
PV Planned value	Budgeted cost of work **scheduled**	What should have been done by now?
EV Earned value	Budgeted cost of work **performed**	What has actually been done?
AC Actual cost	Cost incurred doing it	What did it cost?

Every other EVM quantity is a comparison of two of these three. EV against AC gives cost performance; EV against PV gives schedule performance. The whole method follows from having the third number.

1.2 Measures implemented

Variances. CV = EV − AC. SV = EV − PV. Negative is unfavourable throughout. This convention is worth stating because some contractor reporting formats invert it.

Indices. CPI = EV / AC, the rupees of budgeted work returned per rupee spent. SPI = EV / PV, the work done per unit of work scheduled. Below 1.00 is unfavourable.

Forecasts. Four standard estimate-at-completion methods, which differ in exactly one assumption — what is believed about the cost performance of the work that remains:

Method	Formula	Assumption
EAC1	BAC / CPI	Performance to date is typical and continues
EAC2	AC + (BAC − EV)	The variance was a one-off; the rest runs at budget
EAC3	AC + (BAC − EV) / (CPI × SPI)	Both cost and schedule pressure continue
EAC4	AC + (BAC − EV) / (0.8·CPI + 0.2·SPI)	Cost dominates, schedule partly weighted

From these: ETC = EAC − AC, VAC = BAC − EAC, and TCPI = (BAC − EV) / (EAC − AC), the efficiency the remaining work must achieve to hit a given target.

The dashboard presents all four together rather than choosing one. A single EAC implies a precision the method does not have. On the case study the spread between the most and least optimistic is ₹3.47 crore on a ₹14 crore project, and that spread *is* the honest answer: the method cannot narrow it without a judgement about whether the rock excavation overrun was a closed, one-off event.

1.3 Earned Schedule, and why we added it

Classical schedule variance has a defect that matters most on exactly the projects where schedule control matters most.

SV and SPI are measured in **currency**, not time. At completion, EV necessarily equals BAC and PV necessarily equals BAC. Therefore SV returns to zero and SPI returns to 1.00 — on every project, however late it finished. The indices improve through the final third of a project, and the improvement is an artefact of the arithmetic rather than a recovery. A planner relying on SPI alone receives his most reassuring reading at the moment the project is latest.

Earned Schedule (Lipke, 2003) asks a different question. Rather than "how much value has been earned compared with the value scheduled", it asks "**at what point on the baseline was this much value scheduled to have been earned**". That point is a time, so the resulting measures are in time:

```
ES      the baseline time at which cumulative PV equals current EV
AT      actual time elapsed at the data date
SV(t) = ES − AT        schedule variance, in periods
SPI(t) = ES / AT       time-based schedule efficiency
```

On the case study at Period 6, classical SPI reads **0.863** and SPI(t) reads **0.893**. More usefully, SV(t) is **−0.64 months**, which the dashboard states as "about 19 days behind programme". A site manager can act on nineteen days. Nobody can act on 0.863.

We implement **both** and display both, with the gap explained, rather than silently preferring one. The interpolation assumes planned value accrues linearly within a reporting period — a mild assumption over a monthly period on building work, and one that would not be safe on very long periods or very lumpy spend.

1.4 How earned value is measured

By the **percent-complete method** at work-package level:

```
EV(package) = cumulative percent complete × budget at completion of that package
```

This is the appropriate default for building work, where progress is measured as quantity executed against quantity in the bill. The alternatives — 0/100, 50/50 and milestone weighting — are better suited to short packages and to procurement, and would have forced an arbitrary choice of milestone for packages like "RCC beams and slabs, ground to fourth floor" that genuinely do progress continuously.

Its weakness is that the percent is a judgement. See section 5.

1.5 Why work-package level rather than a rolled-up curve

A single project S-curve can tell a manager that something is wrong. It cannot tell them where, and "the project CPI is 0.87" is not an actionable statement.

Carrying the WBS through the whole calculation means the tool can say instead: the ₹51.20 lakh adverse cost variance is **74% attributable to three substructure packages**, all of which are now complete, so that portion of the variance is final and cannot be recovered; while the superstructure packages that are still running are at CPI 0.886 and 0.891 and *can* still move. That is a different conversation, and it is the one worth having.

This is the main functional difference between this tool and the rolled-up S-curve builders we looked at as prior art (section 3).

---

2. Case study data

2.1 What it is

Project	Academic Block (G+4 RCC), institutional campus
BAC	₹14,00,00,000
Baseline	12 monthly periods
Work packages	19 leaves across 6 WBS branches
Data date	End of Period 6
Position at data date	CPI 0.872, SPI 0.863, SPI(t) 0.893, 24.8% complete

The underlying narrative the figures express:

- Hard rock was encountered in excavation (WBS 2.1) from the second week. Pneumatic breaking and controlled blasting were needed for roughly 60% of the excavated volume, against an estimate assuming ordinary soil. The package finished in Period 5 against a baseline of Period 3 and cost ₹61.0 lakh against a budget of ₹42.0 lakh — CPI 0.689.
- The delay cascaded into footings and foundation RCC, both of which overran and finished late.
- Superstructure therefore started a month late and is running an 18-day slab cycle against a planned 12 days.
- Staircase and lift core RCC has not started. Nothing downstream of the frame has begun.

2.2 Why synthetic data

This was a deliberate choice and it has a cost, so it should be defended.

Why not real project data. Actual cost data at work-package level is commercially sensitive. A contractor's real CPI by package reveals margin structure, and obtaining a genuine time-phased baseline with matched actuals would have required a confidentiality agreement that a four-week student project cannot support. Anonymising it properly — scaling values, renaming packages — would have degraded it to something not meaningfully more real than a well-constructed synthetic set.

What synthetic data buys. Complete control over what the case demonstrates. The project is built to exercise every branch of the method: a package that is complete and over budget (variance final, unrecoverable), a package in progress and over budget (still movable), a package not yet started, a

schedule slip that cascades, and a CPI that stabilises early and stays flat — the empirical regularity that most justifies distrusting an optimistic recovery plan.

What it costs. We cannot claim the tool has been tested against the messiness of real data: missing periods, retrospective corrections, packages whose scope changed mid-project, costs booked to the wrong code. Section 6 of the reflection report treats this as the main gap between this work and something a contractor could deploy.

Honesty about the figures. Budgets are order-of-magnitude plausible for an institutional building of this size built to CPWD DSR-style item rates. They are **assumed**. They are not taken from any tender, estimate or invoice, and no claim is made that they are accurate unit rates.

2.3 How the baseline is phased

Each work package has a baseline start, a baseline finish and a curve shape. Linear packages accrue budget evenly. S-shaped packages follow smoothstep, $3t^2 - 2t^3$, which has zero slope at both ends so a package ramps up and tails off rather than starting and stopping abruptly. That is the usual shape of a construction work package, and aggregating nineteen of them staggered across twelve months is what produces the project S-curve — the curve is an emergent property of the phasing, not a shape imposed on the data.

Generated by `data/generate_case_study.py`, which is deterministic and documented. Rerunning it reproduces the CSVs exactly.

---

3. Prior art

3.1 What we looked at

`Tahir-yamin/excel-scurve-builder` (MIT licence, https://github.com/Tahir-yamin/excel-scurve-builder) — generates Excel S-curve and earned value workbooks from progress CSVs.

We read this repository before designing ours and took three things from it. None of its code was copied; what we took were ideas and hard-won operational knowledge, and it is credited here and in the source files where each is applied.

1. Live formulas rather than pasted values. Its central idea is that the generated workbook is a working tool — type a new contract value and the whole book recalculates — not a static report. We adopted this for both the manual benchmark workbook and the tool's Excel export. It matters because a report a quantity surveyor cannot re-run is a report they will retype, and retyping is where errors enter.

2. The verification argument. Its documentation makes a point we had not considered and that reshaped our validation strategy:

> a self-check that re-implements the intended logic in Python only validates your own understanding — it will agree with your bug.

This is the reason `scripts/validate_benchmark.py` drives the benchmark workbook through **LibreOffice** and reads back what a real spreadsheet engine computed, rather than re-implementing the

Excel formulas in Python and comparing. Had we done the latter, a formula we had misunderstood would have been misunderstood identically on both sides and the validation would have passed.

3. Excel chart defaults that silently produce broken charts. Its `docs/excel-chart-gotchas.md` documents several OOXML defaults that render a broken chart while the XML looks correct: date-like category labels promoting the axis to a date axis and collapsing bars to hairlines; `invertIfNegative` defaulting to true and filling negative bars white; tick labels defaulting to the axis crossing, which on all-negative data is the top of the plot; and a formula returning `""` being plotted as zero rather than skipped. All four apply directly to our variance and S-curve charts and all four are handled explicitly in `evm/excel_export.py`, with a test asserting each override is in place.

Where we went further. That tool rolls each project up to a single percentage curve and lists the absence of a WBS breakdown as a known limitation. Work-package-level EV is the main thing our tool adds, along with Earned Schedule and the four-method forecast range.

3.2 Standard references

- PMI, *A Guide to the Project Management Body of Knowledge*, for the standard definitions of PV, EV, AC, the variances, the indices and the four EAC methods.
- W. Lipke, "Schedule is Different", *The Measurable News*, 2003 — the original statement of Earned Schedule and of the defect in classical SPI that it corrects.
- The empirical regularity that CPI stabilises early in a project, often by the 20% complete point, and rarely recovers. This is widely reported in the earned value literature on large defence and infrastructure programmes. We use it as the justification for treating a flat, below-1.00 CPI as strong evidence against an optimistic recovery plan, and we have stated it as a reported finding rather than something this project has independently verified.

---

4. Data source list

Source	Used for	Status
`data/generate_case_study.py`	The entire case study: WBS, budgets, baseline phasing, reported progress and actual cost	**Synthetic.** Authored for this project. Deterministic and reproducible.
CPWD DSR-style item rates (general knowledge of the rate structure)	Sanity-checking the order of magnitude of work package budgets	**Not quoted.** No rate is reproduced. Used only to judge whether ₹2.10 crore for the RCC frame of a G+4 block is plausible.
PMBOK standard definitions	Formulas for PV, EV, AC, CV, SV, CPI, SPI, EAC 1–4, ETC, VAC, TCPI	Reference text
Lipke (2003)	Earned Schedule, SPI(t), SV(t), IEAC(t)	Reference paper

Source	Used for	Status
`Tahir-yamin/excel-scurve-builder` (MIT)	Live-formula workbook approach; validation-by-execution argument; Excel chart defaults	Prior art, credited. No code reused.

No external dataset was downloaded, scraped or incorporated. The project has no data licensing or privacy exposure, which is itself a consequence of the synthetic-data decision in section 2.2.

---

5. The limitation that governs the method

Everything above describes arithmetic. The arithmetic has been validated to nine decimal places against an independent implementation.

None of it addresses the input. **Earned value is percent complete multiplied by budget, and percent complete is a judgement made at site by a person who may have an interest in the answer.** A package reported at 60% when it is really 40% produces an earned value wrong by lakhs, and the tool will report the resulting CPI with six-figure precision and a green tick where it belongs.

No spreadsheet, no dashboard and no amount of validation can detect this. Only a measurement at site can. The practical consequence for anyone deploying such a tool is that the governance around *how percent complete is assessed and by whom* matters more than the software, and is the part most likely to be neglected because it is the part that cannot be bought.

---

6. On the use of AI in this project

The project was built with AI assistance throughout, logged in full in `docs/05-ai-usage-log.md`. One methodological decision deserves stating here because it shaped the tool.

The monthly commentary is generated from templates and thresholds, deterministically. It does not call a language model. This was a deliberate choice, not a limitation we failed to overcome, for three reasons:

1. **Reproducibility.** The same inputs must produce the same sentence every month. "The model phrased it differently this time" is not a defence in front of a client.
2. **It cannot invent.** A language model given a table of indices will readily produce a fluent paragraph containing a number that is not in the table. In the commentary module there is no path by which a figure can reach the text without having been computed first, and a test asserts it.
3. **It is auditable.** Every sentence traces to a threshold written down in `evm/commentary.py` and justified there. A reader who disagrees with the "within 5% is noise" band can find it and argue with it.

Where AI genuinely earned its place was upstream: drafting those templates, checking formula logic, generating code, and catching an arithmetic slip in our own hand calculation. Not in generating the numbers a decision rests on. That distinction is this project's main argument about AI in professional practice and is developed in the reflection report.

CE725 CONSTRUCTION ECONOMICS AND FINANCE

Topic 7 — Earned Value Management Dashboard

User Manual

Team 40

Sajjad Nakhwa	23B0702	Project Lead and Finance Modeller
Dependar Arya	23B0331	Data Lead
Neel Shah	23B4244	AI and Tool Developer
Akshat Karkar	23B4229	Validation Engineer
Pearsh Gumasta	23B4232	Documentation and Reflection Lead

Semester course project · November 2026

Version 1.0 · For project managers, planning engineers and cost consultants

Live tool: https://ce725-evm-dashboard-3hfqkptabnvbzvzyb8uopr.streamlit.app

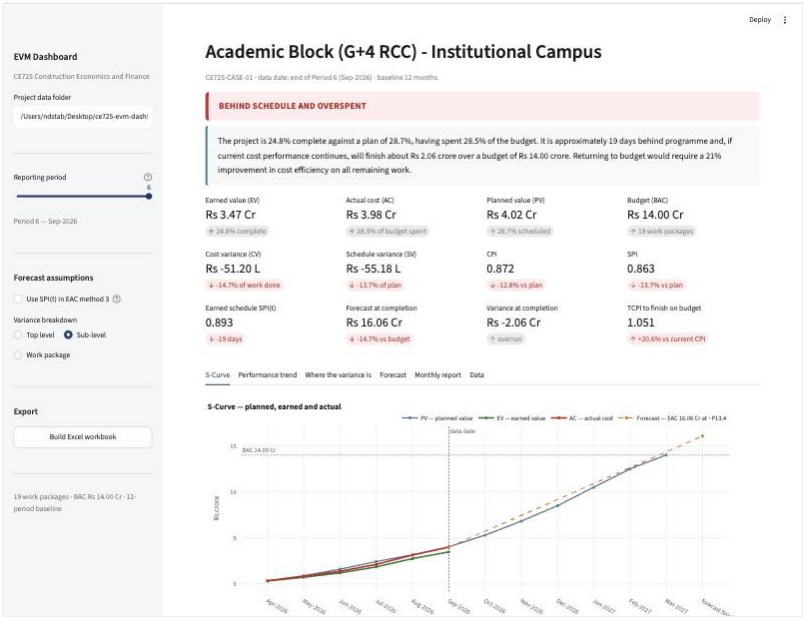
---

1. What this tool does

It takes a project's budget, its baseline programme and its reported monthly progress and cost, and answers three questions:

1. **Where the project stands** — over budget, behind programme, or both, and by how much in rupees and in days.
2. **Where the problem is** — which work packages drive the variance, and whether that variance is still recoverable.
3. **Where it will end up** — forecast cost under four assumptions, and whether the original budget is still reachable.

It is a reporting and diagnostic tool. It does not plan, schedule or price work.



2. Starting the tool

Open the live link above — it needs nothing installed and ships with a worked case study, so it is usable immediately, and you can load your own project into it from the sidebar (section 3). To run it locally instead (Python 3.11+):

```
git clone https://github.com/ndstab/ce725-evm-dashboard.git
cd ce725-evm-dashboard && python -m venv .venv
.venv/bin/pip install -r requirements.txt
.venv/bin/streamlit run app/streamlit_app.py
```

3. Required inputs

The tool runs on any project supplied as three CSVs and one JSON. In the sidebar, **Download data template** gives you the worked case study in exactly the format expected — edit those four files and load them back through **Upload your own**. A working example is easier to edit correctly than a blank form is to fill in, because the baseline has to reconcile to each package budget and the packages to BAC, which is easier to preserve than to achieve.

`wbs.csv` — the work breakdown structure. Columns: `wbs_id` (dotted for hierarchy: `2`, `2.1`), `parent_id` (blank at top level), `name`, `is_leaf` (`Y` carries budget, `N` is a summary branch), `budget_inr` (leaves only). Only leaves carry budget; a branch is the sum of its leaves. Budget a branch *and* its children and you have double-counted — the tool rejects the file.

`baseline.csv` — the time-phased budget. Columns: `wbs_id`, `period` (1 upward), `pv_period_inr`. This is the money you planned to earn, period by period. Per package it must sum to that package's budget, and overall to BAC.

`actuals.csv` — what happened. Columns: `wbs_id`, `period`, `pct_complete_cum`, `ac_period_inr`. Note the asymmetry, which is the commonest source of error: progress is **cumulative and a fraction** (`0.45`, not `45`), cost is **per period**. That is how each is normally reported — site measures progress to date, accounts book a month's cost — so each is taken in its natural form.

`project.json` — constants. `project_name`, `project_code`, `currency`, `baseline_periods`, `period_unit`, `data_date_period`, `period_labels`, `bac_inr`. `data_date_period` is the last period with reported actuals; nothing after it may appear in `actuals.csv`.

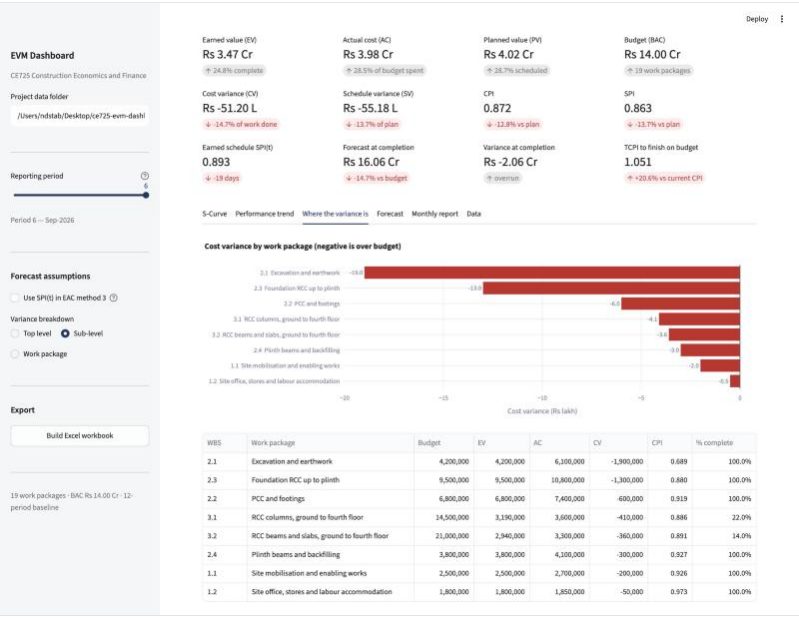
Input is validated strictly, and bad data is **refused rather than computed from**. EVM output is three ratios near 1.00 with no sanity check a reader can apply: a CPI of 0.94 from corrupt data looks exactly like a CPI of 0.94 from good data, and it will be taken to a client meeting. Each rejection message names the file, the row and the fix.

4. Reading the dashboard

The verdict and summary. The coloured bar is a traffic light. The paragraph under it is written for someone who will not read the rest, and quotes no index without the consequence in rupees or days beside it. It is produced from templates, not a language model: the same inputs give the same sentence every month, and every threshold behind a phrase like "behind plan" is written in `evm/commentary.py` where you can disagree with it.

The index tiles. CPI is rupees of budgeted work returned per rupee spent — 0.872 means ₹1 buys 87 paise of budgeted work. SPI is the same question for schedule, measured in money; SPI(t) measures it in time. EAC forecasts total cost, VAC the overrun (negative is bad), and TCPI the efficiency the *remaining* work must achieve to finish on budget.

The most useful single comparison is TCPI against CPI. On the case study the project has sustained CPI 0.872 for six months while TCPI demands 1.051 — a 20.6% improvement on performance never once achieved. The budget is gone; what remains is deciding who absorbs it.



WBS	Work package	Budget	EV	AC	CV	CPI	% complete
2.1	Excavation and earthwork	4,200,000	4,200,000	6,100,000	-1,900,000	0.689	100.0%
2.3	Foundation RCC up to plinth	9,500,000	9,500,000	10,800,000	-1,300,000	0.880	100.0%
2.2	PCC and footings	6,800,000	6,800,000	7,400,000	-600,000	0.919	100.0%
3.1	RCC columns, ground to fourth floor	14,500,000	3,190,000	3,600,000	-410,000	0.886	22.0%
3.2	RCC beams and slabs, ground to fourth floor	21,000,000	2,940,000	3,300,000	-360,000	0.891	14.0%
2.4	Plinth beams and backfilling	3,800,000	3,800,000	4,100,000	-300,000	0.927	100.0%
1.1	Site mobilisation and enabling works	2,500,000	2,500,000	2,700,000	-200,000	0.926	100.0%
1.2	Site office, stores and labour accommodation	1,800,000	1,800,000	1,850,000	-50,000	0.973	100.0%

Where the variance is. Bars run left for over budget. Percent complete is the column that decides what you can do: a package at **100%** has a variance that is **final**, and no recovery plan touches it; a package part complete can still

move. On the case study ₹38 lakh of the ₹51.20 lakh adverse variance sits in three finished substructure packages. Only ₹7.7 lakh, in the frame, is still live.

Performance trend. Watch two things. *CPI flatness* — CPI stabilises early, often by 20% complete, and rarely recovers; here it holds between 0.86 and 0.88 for all six periods while SPI swings from 0.88 to 0.74 and back. A flat, below-1.00 CPI is the strongest argument against an optimistic recovery plan. *The SPI / SPI(t) gap* — at completion EV must equal PV, so **classical SPI returns to 1.00 on every project however late it finishes**, growing more reassuring as the project gets later. SPI(t) is measured in time and has no such defect. Late in a project, trust SPI(t).

Forecast. Four EAC methods, shown together on purpose. They differ in one assumption — what you believe about the remaining work. **EAC1** (BAC/CPI) assumes nothing has changed, and is hard to beat once CPI has settled. **EAC2** assumes the overrun had an identified, now-closed cause. **EAC3** and **EAC4** assume schedule pressure continues, and acceleration costs money. A single EAC implies a precision the method does not have. Quote the range.

Monthly report. A generated narrative for pasting into a progress report, downloadable as text.

5. Exporting to Excel

Build Excel workbook, then **Download workbook**. Every figure is a **live formula**, not a pasted value: change a budget or a percent complete on the yellow Inputs sheet and the indices, forecasts, verdict and charts all recalculate. Hand it to a quantity surveyor who has never seen this tool and they can work with it. Sheets: Dashboard, Inputs, Baseline, EVM, Variance.

6. Key assumptions

1. **Percent complete is accurate** — see section 7.
2. **Planned value accrues linearly within a period**, used when interpolating Earned Schedule. Mild over a month; unsafe over a quarter.
3. **One baseline.** Re-baselining is not modelled.
4. **Work continues.** Forecasts extrapolate performance; a suspended project needs a re-baseline, not an extrapolation.
5. **Nominal rupees at one price base.** No escalation, no multi-currency.
6. **Costs are booked to the right package in the right period.** The tool cannot detect miscoding.

7. Known limitations

The one that matters most. Earned value is percent complete multiplied by budget, and percent complete is a judgement made at site, often by someone with an interest in the answer. **If a package is reported at 60% when it is really 40%, every index here is confidently, precisely wrong, and nothing in the software can detect it.** The arithmetic has been validated to nine decimal places against an independent benchmark (see `docs/04-validation-report.md`). That validates the *calculation* and says nothing about the *measurement*. Deploy a tool like this and the governance around how percent complete is assessed, and by whom, matters more than the software — and it is the part most likely to be neglected, because it is the part you cannot buy.

Others, in order of how likely they are to bite:

- **No critical path.** The tool reads a time-phased budget, not a network. It can say the project is 19 days behind in aggregate; it cannot say whether the slipped package is critical, so it cannot say whether the completion date has moved.
- **No re-baselining.** One baseline per project.
- **Equal-length periods assumed.** Mixing a two-week period with a five-week one distorts the trend charts.
- **No retention, advance or cash flow.** A project can be green on every index here and still run out of money.
- **Variations assumed already in the budget.** If scope was added and the budget not revised, packages report above 100% complete and the file is rejected rather than guessed at.

- **Not tested against messy real data.** The case study is synthetic. Missing periods, retrospective corrections and miscoded costs are handled in principle, not against a real ledger.

Do not take the arithmetic on trust. `pytest tests/` runs 151 tests whose expected values were hand-derived from the case study before the engine existed; `python scripts/validate_benchmark.py` re-runs the full comparison against the independent Excel benchmark (needs LibreOffice).

CE725 CONSTRUCTION ECONOMICS AND FINANCE

Topic 7 — Earned Value Management Dashboard

Validation Report

Team 40

Sajjad Nakhwa	23B0702	Project Lead and Finance Modeller
Dependar Arya	23B0331	Data Lead
Neel Shah	23B4244	AI and Tool Developer
Akshat Karkar	23B4229	Validation Engineer
Pearsh Gumasta	23B4232	Documentation and Reflection Lead

Semester course project · November 2026

Benchmark case: reporting periods 5 and 6 of the case study project

---

1. Summary

Measures compared	25 per period, 2 periods, **50 comparisons**
Agreement	**50 of 50**, within tolerance
Tolerance	₹0.01 on currency, 1×10^{-9} on dimensionless indices
Discrepancies found during development	6, all resolved — section 5
Automated tests	**151**, all passing

The headline result is that two independently written implementations of the EVM definitions — one in Python, one in typed Excel formulas — agree on every measure, at both reporting periods, to nine decimal places.

What that does and does not establish is set out in section 6, and matters more than the result itself.

---

2. Validation strategy

2.1 The approach, and the trap it avoids

The obvious way to validate a calculation engine is to write a second implementation in the same language and compare. We did not do that, and the reason reshaped the whole approach.

A Python script that re-implements the intended Excel logic and compares it with the engine is **checking our understanding against itself**. If we misread a formula — used EV/PV where the definition says EV/AC, or applied TCPI against the wrong denominator — the re-implementation embeds the same misreading, agrees perfectly with the engine, and the validation passes while both are wrong. The test would be measuring our consistency, not our correctness.

This argument is taken from the verification notes in `Tahir-yamin/excel-scurve-builder`, which states it as: *a self-check that re-implements the intended logic in Python only validates your own understanding — it will agree with your bug.*

So the benchmark is built in a **different execution environment with different semantics**, and it is **executed rather than re-implemented**:

```
Raw inputs  (budgets, % complete, actual cost, time-phased budget)
     |
     +---------------------------+
     |                           |
Python engine              Excel formulas
(evm package)       (EVM_Manual_Benchmark.xlsx)
     |                           |
     |                    evaluated by
     |                  LibreOffice headless
     |                           |
     +-----------> compare <-----+
                 50 measures
```

`scripts/validate_benchmark.py` drives LibreOffice over the workbook, which loads it, recalculates every cell with a real spreadsheet engine, and writes the computed values back. We then read those values. At no point does Python compute what the Excel formula "should" give.

2.2 What is in the benchmark workbook

`benchmark/EVM_Manual_Benchmark.xlsx` contains, for periods 5 and 6:

- **Inputs sheet** — the only typed numbers in the workbook: 19 work package budgets, actual cost for six periods, percent complete for six periods.
- **Baseline sheet** — the time-phased budget, with a reconciliation column asserting each package's phasing sums to its budget, and a cumulative planned value row that Earned Schedule interpolates along.
- **Calc P5** / **Calc P6** — per work package: `EV = budget × % complete`, cumulative PV, cumulative AC, CV, SV, CPI, SPI.
- **Indices sheet** — all 25 measures, as formulas, both periods side by side.

Every green cell is a formula. Open the file, click any of them, and the arithmetic is visible.

On the workbook being script-placed. The formulas are laid out by `benchmark/build_benchmark.py` rather than typed cell by cell. 19 packages across 12 baseline periods is 228 planned-value cells, and hand-typing them would introduce exactly the transcription errors the benchmark exists to catch. The script places formulas; Excel evaluates them. **The formulas are the manual method** — no value is carried across from Python.

---

3. The hand calculation

Both periods were worked by hand from the raw inputs before the engine was run. Reproduced here so the arithmetic can be followed without opening anything.

3.1 Earned value — Period 6

`EV = cumulative percent complete × budget at completion`

WBS	Work package	Budget (₹ lakh)	% complete	EV (₹ lakh)
1.1	Site mobilisation and enabling works	25.00	100%	25.00
1.2	Site office, stores and labour accommodation	18.00	100%	18.00

WBS	Work package	Budget (₹ lakh)	% complete	EV (₹ lakh)
2.1	Excavation and earthwork	42.00	100%	42.00
2.2	PCC and footings	68.00	100%	68.00
2.3	Foundation RCC up to plinth	95.00	100%	95.00
2.4	Plinth beams and backfilling	38.00	100%	38.00
3.1	RCC columns, ground to fourth floor	145.00	22%	31.90
3.2	RCC beams and slabs, ground to fourth	210.00	14%	29.40
3.3	Staircase and lift core RCC	55.00	0%	0.00
—	All packages downstream of the frame	609.00	0%	0.00
	Total EV	**1,400.00**		**347.30**

EV = ₹3,47,30,000

3.2 Actual cost — Period 6

Cumulative sum of cost booked per period, ₹ lakh:

WBS	P1	P2	P3	P4	P5	P6	Total	Budget	Variance
1.1	15.0	12.0	—	—	—	—	27.0	25.0	−2.0
1.2	9.5	9.0	—	—	—	—	18.5	18.0	−0.5
2.1	11.0	15.0	16.5	11.0	7.5	—	**61.0**	42.0	**−19.0**
2.2	—	11.0	19.5	25.0	18.5	—	74.0	68.0	−6.0
2.3	—	—	19.0	30.0	34.0	25.0	**108.0**	95.0	**−13.0**
2.4	—	—	—	8.5	16.0	16.5	41.0	38.0	−3.0
3.1	—	—	—	—	13.0	23.0	36.0	—	—
3.2	—	—	—	—	12.0	21.0	33.0	—	—
							398.5		

AC = ₹3,98,50,000

3.3 Planned value — Period 6

Packages whose baseline finish is Period 5 or earlier are fully planned:

```
1.1 25 + 1.2 18 + 2.1 42 + 2.2 68 + 2.3 95 + 2.4 38  =  286.000 lakh
```

Packages still in progress against the baseline, phased by smoothstep $3t^2 - 2t^3$ where S-shaped:

```
3.1  P5–P9,  n = 5, 2 elapsed, t = 0.4
     3(0.16) − 2(0.064) = 0.352          145.00 × 0.352   =   51.040
3.2  P5–P10, n = 6, 2 elapsed, t = 1/3
     3(1/9) − 2(1/27)  = 7/27            210.00 × 7/27    =   54.444
3.3  P6–P10, n = 5, linear, 1 elapsed    55.00 × 0.20     =   11.000
                                                            ---------
                                                Total       402.484 lakh
```

PV = ₹4,02,48,444

3.4 Variances, indices and forecasts — Period 6

```
CV   = EV − AC = 3,47,30,000 − 3,98,50,000        = −51,20,000
SV   = EV − PV = 3,47,30,000 − 4,02,48,444        = −55,18,444
CPI  = EV / AC = 3,47,30,000 / 3,98,50,000        =  0.871518
SPI  = EV / PV = 3,47,30,000 / 4,02,48,444        =  0.862890
```

Earned Schedule. Cumulative PV reaches ₹3,16,63,556 at Period 5 and ₹4,02,48,444 at Period 6; current EV of ₹3,47,30,000 falls between them:

```
ES     = 5 + (3,47,30,000 − 3,16,63,556) / (4,02,48,444 − 3,16,63,556)
       = 5 + 30,66,444 / 85,84,889
       = 5.357191 months
SV(t)  = 5.357191 − 6 = −0.642809 months   ≈ 19 days behind
SPI(t) = 5.357191 / 6 =  0.892865
```

Forecasts, with remaining work BAC − EV = ₹10,52,70,000:

```
EAC1 = BAC / CPI                      = 14,00,00,000 / 0.871518    = 16,06,39,217
EAC2 = AC + (BAC − EV)                = 3,98,50,000 + 10,52,70,000 = 14,51,20,000
EAC3 = AC + (BAC − EV)/(CPI × SPI)    ÷ 0.7520248                  = 17,98,32,093
EAC4 = AC + (BAC − EV)/(0.8CPI+0.2SPI) ÷ 0.8697857                 = 16,08,78,845

TCPI(BAC) = (BAC − EV)/(BAC − AC) = 10,52,70,000 / 10,01,50,000    = 1.051123
```

3.5 Period 5, in brief

Worked the same way:

```
EV = 25 + 18 + 42 + 68 + (0.80×95) + (0.62×38) + (0.08×145) + (0.05×210)
   = 25 + 18 + 42 + 68 + 76.00 + 23.56 + 11.60 + 10.50  =  274.66 lakh
AC = 27 + 18.5 + 61 + 74 + 83 + 24.5 + 13 + 12          =  313.00 lakh
PV                                                       =  316.636 lakh

CPI = 274.66 / 313.00    = 0.877508
SPI = 274.66 / 316.636   = 0.867433
ES  = 4 + (274.66 − 242.370) / (316.636 − 242.370) = 4.434788
SPI(t) = 4.434788 / 5    = 0.886958
```

---

4. Comparison result

All 50 comparisons agree. The complete machine-generated table is at `generated/benchmark-comparison.md`; the headline measures:

Period 6

Measure	Tool (Python)	Manual benchmark (Excel)	Difference
Planned value (PV)	40,248,444.43	40,248,444.43	0.00
Earned value (EV)	34,730,000.00	34,730,000.00	0.00
Actual cost (AC)	39,850,000.00	39,850,000.00	0.00

Measure	Tool (Python)	Manual benchmark (Excel)	Difference
Cost variance (CV)	−5,120,000.00	−5,120,000.00	0.00
Schedule variance (SV)	−5,518,444.43	−5,518,444.43	0.00
CPI	0.871518193	0.871518193	0.000000000
SPI	0.862890492	0.862890492	0.000000000
Earned schedule (ES)	5.357190932	5.357190932	0.000000000
SPI(t)	0.892865155	0.892865155	0.000000000
SV(t)	−0.642809068	−0.642809068	0.000000000
EAC method 1	160,639,216.82	160,639,216.82	0.00
EAC method 2	145,120,000.00	145,120,000.00	0.00
EAC method 3	179,832,092.74	179,832,092.74	0.00
EAC method 4	160,878,844.80	160,878,844.80	0.00
TCPI to BAC	1.051123315	1.051123315	0.000000000

Period 5

Measure	Tool (Python)	Manual benchmark (Excel)	Difference
Planned value (PV)	31,663,555.54	31,663,555.54	0.00
Earned value (EV)	27,466,000.00	27,466,000.00	0.00
Actual cost (AC)	31,300,000.00	31,300,000.00	0.00
CPI	0.877507987	0.877507987	0.000000000
SPI	0.867432590	0.867432590	0.000000000
Earned schedule (ES)	4.434788249	4.434788249	0.000000000
SPI(t)	0.886957650	0.886957650	0.000000000
EAC method 1	159,542,707.35	159,542,707.35	0.00
EAC method 3	179,141,698.34	179,141,698.34	0.00
TCPI to BAC	1.035271389	1.035271389	0.000000000

4.1 Algebraic identities, checked on both sides

Two identities hold only if the entire calculation chain is intact. They are asserted against the Python engine *and* against the independently computed Excel figures, so agreement is not an artefact of one implementation being written to satisfy them.

Identity	Why it must hold	Result
TCPI against EAC1 = CPI	Substituting EAC = BAC/CPI into TCPI = (BAC−EV)/(EAC−AC) reduces to CPI exactly. A forecast built on "performance continues" demands, by construction, no change in performance.	Holds in both, to 1e−9

Identity	Why it must hold	Result
VAC on EAC2 = CV	$VAC_2 = BAC - (AC + BAC - EV) = EV - AC = CV$. If the rest runs at budget, the final overrun is exactly the overrun already incurred.	Holds in both, to ₹0.01
TCPI against EAC2 = 1.000	EAC2 assumes the remainder runs at budget, so it requires efficiency of exactly 1.00.	Holds exactly

---

5. Discrepancies found, and how each was resolved

Four genuine problems surfaced during validation. All are recorded, including the ones that turned out to be our own error rather than the tool's.

5.1 EAC method 3 differed by ₹156 — rounding in the calculation chain

Found. The hand calculation gave EAC3 = ₹17,98,32,249. The engine gave ₹17,98,32,093. A difference of ₹156.

Investigated. Rather than assume either side was right, the quantity was recomputed with 40-digit decimal arithmetic:

```
CPI × SPI, full precision     0.752024761995748023233 6676...
CPI × SPI, rounded to 6 dp    0.752025
EAC3 from full precision      179,832,092.77
EAC3 from the rounded index   179,832,048.47
```

Cause. The hand calculation had rounded the composite index CPI × SPI to six decimal places mid-chain — which is what one naturally does on paper, and looks harmless. It is not: because EAC3 divides a ₹10.5 crore remainder by that index, a change in the seventh decimal place moves the forecast by tens of rupees. Our cruder rounding moved it by ₹156.

Resolved. The engine was correct. The hand figure was corrected. More usefully, a rule was adopted for the rest of the work: **rounding belongs in presentation only, never in the calculation chain.** The benchmark workbook applies rounding through cell number formats alone, and the engine never rounds. A regression test (`test_rounding_an_intermediate_index_moves_the_forecast`) pins the lesson so it cannot quietly return.

Why it is worth reporting despite being immaterial. ₹156 on a ₹18 crore forecast is nothing. But the same habit applied to CPI itself early in a project, when EV is small and the divisor is volatile, moves the forecast by lakhs. The discrepancy was trivial; the practice behind it was not.

5.2 Two validation tests were mis-constructed, not the engine

Found. Two tests of the input validation rules failed, apparently showing the wrong error being raised.

Investigated.

- `test_package_budgets_must_sum_to_the_declared_bac` mutated a work package budget. That broke per-package reconciliation, which is checked *earlier*, so that error fired first.

- `test_actuals_may_not_run_past_the_data_date` appended a row copied from Period 1, carrying a percent complete lower than the package had already reached. That tripped the monotonicity rule first.

Resolved. In both cases the engine's behaviour was correct and the test was wrong. The tests were rewritten to exercise each rule in isolation — the first by altering the declared BAC rather than a package budget, the second by carrying progress forward rather than reversing it. **No engine code was changed.**

Why it is reported. It is the failure mode the brief warns about, in miniature: a test failed, and the tempting move is to "fix" the code until the test passes. Doing so here would have *removed* a correct validation rule to satisfy a badly written test.

5.3 Executive summary invisible under a dark browser theme

Found. Only by looking at the running application. The custom CSS set light panel backgrounds without setting a text colour, so when the browser requested dark mode Streamlit supplied white text, and the executive summary rendered white-on-white — an empty box where the most important paragraph on the screen should be.

Resolved. Explicit `color` on every styled panel, and the theme pinned to light in `.streamlit/config.toml`, with the reasoning recorded there: the figures get screenshotted into progress reports and printed, and a dark dashboard prints as a black rectangle.

Why it is reported. No unit test would have caught it. The numbers were all correct; the tool was simply unusable. It is a reminder that "validated" and "works" are different claims.

5.4 Progress carry-forward logic was rewritten before it could fail

Found. In review rather than by a failing test. The routine filling gaps in reported progress had grown a nested loop with a conditional that indexed a pivot table by a column that might not exist.

Resolved. Replaced with a forward-fill along the period axis followed by a zero-fill. The semantics are now stated plainly in the code: a missing period means progress is unchanged, and anything before the first report is zero.

Why it is reported. It had not produced a wrong answer, because the case study has no gaps in reporting. Real project data does. The defect would have surfaced in exactly the situation the tool is meant to handle.

5.5 A project under budget was told to improve by –5%

Found. By building a second case study specifically to exercise the branches the first one never reaches. The bundled project is adverse on both cost and schedule, so every favourable path through the commentary was unexercised. A second synthetic project — a 15 MLD water treatment plant, behind programme but **under** budget at CPI 1.041 — produced this closing sentence in its executive summary:

> Returning to budget would require a **−5% improvement** in cost efficiency on all remaining work.

Cause. The sentence was generated unconditionally from TCPI divided by CPI. When a project is already forecast to underrun, that ratio is below one and the "improvement required" is negative, which is not a quantity that exists.

Resolved. The closing sentence now branches. A project forecast to overrun is told what improvement is required; a project forecast to underrun is told how much headroom it has — "*the remaining work could run about 5% less cost efficiently than it has so far and still finish within budget*". A regression test (`tests/test_commentary.py::test_under_budget_project_is_not_asked_to_improve`) fixes the behaviour, and the second case study is committed under `samples/case-02-water-treatment/` so the branch stays exercised.

Why it is reported. Every number in the sentence was correct. The arithmetic was right and the English was nonsense, and no comparison against the benchmark could have caught it, because the benchmark checks figures and not prose. It is the same class of defect as 5.3: output that is numerically valid and unusable by a reader. It also makes a narrower point about our own test data — a single case study, however carefully built, only tests the paths it happens to take.

5.6 The Excel export appeared to download an empty file

Found. By a reader opening the exported workbook, which is the only way this could have been found. Every sheet showed its labels and not one number.

Cause. `openpyxl` stores a cell as *either* a formula *or* a value, never both, so every formula we write is saved with no cached result. Real Excel stores both: the formula and the last computed value. We set `fullCalcOnLoad`, which makes Excel and LibreOffice recalculate when the file opens, and there the workbook displays correctly. Anything that reads cached values without calculating — Apple Numbers, macOS Quick Look, most in-browser previewers — finds nothing to read. Inspecting the file confirmed it: 46 populated cells on the Dashboard sheet, of which 29 had cached values, and all 29 were text labels. Passing the same file through LibreOffice headless raised that to 46.

Resolved. The live-formula behaviour is deliberate and was kept — it is the reason the workbook can be interrogated without the tool. What was missing was any way for the reader to know. The Dashboard sheet now carries a note at the top, the Read Me opens with an "*If the dashboard looks empty*" section, and the headline figures are written to the Read Me as plain numbers so that something is always readable. The dashboard's download button repeats the warning.

Why it is reported. The file was valid the entire time. Every formula was correct, and the workbook had been checked by reading it back with the same library that wrote it — which recalculates nothing and so could never have caught this. It took someone opening the file in an ordinary viewer. That is the same lesson as 5.3: we had tested the artefact we produced, not the artefact as it is received.

Found at the same time. Two sidebar controls looked inert. Both in fact worked — the breakdown radio changes the variance table from 6 summary rows to 19 packages, and the SPI(t) checkbox moves EAC3 from ₹17.98 crore to ₹17.51 crore — but each affects exactly one tab, so changing one while reading any other tab does nothing visible. The checkbox was also mislabelled: it reads "EAC method 3" and silently changes EAC4 as well, because EAC4 blends the same index. Each control now names the tab it governs, and the checkbox names both methods.

---

6. What this validation does not establish

This section matters more than section 4.

It validates the calculation. It does not validate the inputs.

The Python engine and the Excel benchmark read the same budgets, the same percent-complete figures and the same costs. That is deliberate — a benchmark against different data would compare nothing. What has been established is that two separately written implementations of the EVM definitions agree. An arithmetic or logic error in one would not be reproduced by the other, and none was found.

But if a work package is reported at 60% complete when it is really 40%:

- the Python engine computes a wrong EV,
- the Excel benchmark computes **the same** wrong EV,
- all 50 comparisons agree perfectly,
- and every index on the dashboard is confidently, precisely wrong.

No spreadsheet and no test suite can detect this. Only a measurement at site can.

The practical consequence for anyone deploying such a tool is that the governance around how percent complete is assessed, by whom, and with what independent check, matters more than the software — and it is the part most likely to be neglected, because it is the part that cannot be bought.

Three further limits of this exercise:

1. **One case study.** The benchmark exercises one project at two data dates. It does not cover re-baselining, gaps in reporting, packages whose scope changed, or negative corrections.
2. **Chart appearance is not machine-checkable.** The tests assert that each Excel chart default known to produce a broken chart has been overridden. Nothing here renders Excel, so whether the chart is *legible* still needs a human to open the file. We opened it.
3. **The data is synthetic.** The tool has not been tested against the messiness of a real cost ledger.

---

7. Reproducing this

```
.venv/bin/python data/generate_case_study.py        # regenerate inputs
.venv/bin/python benchmark/build_benchmark.py       # place the formulas
.venv/bin/python scripts/validate_benchmark.py      # evaluate and compare
.venv/bin/python -m pytest tests/ -v                # 151 tests
```

The third command needs LibreOffice. The test suite does not — it reads a committed snapshot of the LibreOffice-computed values, with a test asserting the snapshot is not older than the workbook it describes.

CE725 CONSTRUCTION ECONOMICS AND FINANCE

Topic 7 — Earned Value Management Dashboard

AI Usage Log

Team 40

Sajjad Nakhwa	23B0702	Project Lead and Finance Modeller
Dependar Arya	23B0331	Data Lead
Neel Shah	23B4244	AI and Tool Developer
Akshat Karkar	23B4229	Validation Engineer
Pearsh Gumasta	23B4232	Documentation and Reflection Lead

Semester course project · November 2026

Format per Annexure A of the course brief.

Declaration

This project was built with substantial AI assistance. The tool used throughout was **Claude (Anthropic), via the Claude Code CLI**, which can read and write files in the repository and run commands, so its output was not only text but working code, spreadsheets and documents.

The log below records the significant interactions. It was maintained during the work, not reconstructed afterwards, and it includes the occasions on which the AI was **wrong**, the occasions on which **we were wrong and the AI was right**, and the occasions on which its output was **rejected on principle** even though it would have worked.

Entries marked **Modify** or **Reject** are the ones worth reading. A log consisting entirely of "Accept" would indicate that nobody was checking.

The **Member** column records which of us ran the interaction and carried out the verification in the final column, assigned by the role allocation in the project charter. Where two of us worked on an entry together the owner of the deliverable is named.

Verification standard applied throughout: no AI-produced number entered a deliverable until it had been reproduced either by hand or by an independent implementation. The 50-comparison benchmark in `docs/04-validation-report.md` is the formal expression of this.

---

1. Log

#	Member	Date	Tool	Purpose / prompt summary	Output summary	Decision	Reason and verification
1	Sajjad	2026-10-29	Claude	Read the course brief; assess which of the 14 topics best fits a team that wants to demonstrate method depth rather than breadth	Recommended Topic 7 (EVM); noted that the brief's own validation requirement ("manually compute all EVM indices for two reporting periods") is unusually well suited to a rigorous two-implementation comparison	**Accept**	Reasoning checked against the brief's assessment weights: validation is 20%, the largest single component alongside tool functionality

#	Member	Date	Tool	Purpose / prompt summary	Output summary	Decision	Reason and verification
2	Sajjad	2026-10-29	Claude	Review `Tahir-yamin/excel-scurve-builder` as prior art and identify what to adopt and what to improve on	Identified three adoptable ideas (live formulas, validation-by-execution, Excel chart defaults) and one clear gap (it rolls up to a single curve and lists the absence of a WBS breakdown as a known limitation)	**Accept**	Read the repository's README and gotchas document directly rather than relying on the summary. Credited in `docs/02-method-note.md` §3.1
3	Dependar	2026-10-29	Claude	Design a case study project that exercises every branch of the EVM method	G+4 RCC academic block, ₹14 crore, 19 work packages, 12 periods, built around a hard-rock excavation overrun cascading into the frame	**Modify**	First draft had the variance spread evenly across packages, which makes the WBS breakdown pointless. Reworked so 74% of the adverse variance sits in three completed substructure packages — the distinction between recoverable and final variance is the thing worth demonstrating
4	Dependar	2026-10-29	Claude	Choose a phasing curve for work packages within their baseline window	Proposed smoothstep $3t^2 - 2t^3$, with zero slope at both ends	**Accept**	Verified the aggregate of 19 staggered packages produces a credible project S-curve, and that cumulative PV lands on exactly ₹14.00 crore. The S-curve is emergent from the phasing, not imposed
5	Neel	2026-10-29	Claude	Implement the core EVM measures in Python	`evm/core.py` with PV/EV/AC matrices and a `PeriodMetrics` dataclass	**Accept**	Smoke-tested against a hand calculation before any further work (entry 9)
6	Neel	2026-10-29	Claude	Review the progress carry-forward routine	Flagged its own earlier code: nested loop with a conditional indexing a pivot table by a column that might not exist	**Modify**	Replaced with forward-fill then zero-fill. Had not yet produced a wrong answer because the case study has no reporting gaps — but real data does. Recorded as discrepancy 5.4 in the validation report
7	Sajjad	2026-10-29	Claude	Explain why classical SPI is unreliable and whether it should be supplemented	Explained the convergence defect (EV → BAC and PV → BAC at completion, so SPI → 1.00 however late) and proposed Earned Schedule	**Accept**	Verified independently: on the case study SPI reads 0.863 and SPI(t) reads 0.893, and SV(t) of −0.64 months converts to a statement a site manager can act on
8	Sajjad	2026-10-29	Claude	Implement the four standard EAC methods	`evm/forecasting.py` with EAC1–4, ETC, VAC, TCPI	**Modify**	Initial version returned a single "best" EAC. Rejected that design: a single EAC implies a precision the method does not have. Changed to return all four with their assumptions stated, and the dashboard presents the range
9	Akshat	2026-10-29	— (manual)	Hand-calculate PV, EV, AC and all indices at Period 6 from the raw inputs	PV ₹4,02,48,444; EV ₹3,47,30,000; AC ₹3,98,50,000; CPI 0.871518; SPI 0.862890	**Accept**	**Done by hand before running the engine.** Engine then matched to six decimal places
10	Akshat	2026-10-29	Claude	Write a test suite whose expected values are the hand figures from entry 9	34 tests in `tests/test_core.py` with the derivations in the docstrings	**Accept**	All passed. Hand derivations written into the tests so the arithmetic can be followed in the viva without re-running anything
11	Akshat	2026-10-29	Claude	Add algebraic identity tests	Proposed TCPI-against-EAC1 = CPI, and VAC-on-EAC2 = CV	**Accept**	Genuinely valuable suggestion we had not thought of. These hold only if the whole forecast chain is intact, so they catch an error that per-value comparison can miss

#	Member	Date	Tool	Purpose / prompt summary	Output summary	Decision	Reason and verification
12	Akshat	2026-10-29	Claude	**Investigate: hand calculation of EAC3 disagrees with engine by ₹156**	Recomputed with 40-digit decimal arithmetic; showed the engine matched full precision and the hand figure had rounded CPI × SPI to six decimal places mid-chain	**Accept**	**The AI was right and we were wrong.** Verified independently with Python's `decimal` module at 40 digits. Corrected the hand figure, adopted "rounding is for presentation only" as a rule, and pinned it with a regression test. Recorded as discrepancy 5.1
13	Akshat	2026-10-29	Claude	Two input-validation tests are failing — diagnose	Determined that in both cases an earlier validation rule fired before the rule under test, so the engine was correct and the tests were mis-constructed	**Accept**	**Resisted the tempting fix.** Changing the engine to make the tests pass would have removed a correct validation rule. Rewrote both tests to exercise each rule in isolation; no engine code changed. Recorded as discrepancy 5.2
14	Sajjad	2026-10-29	Claude	Should the monthly commentary be generated by a language model?	Argued **against** its own use: a cost report must be reproducible month to month, must not contain numbers that were never computed, and must have arguable thresholds	**Accept**	Adopted. Commentary is template-driven and deterministic. A test asserts every figure in the output text is one the engine computed. This is the project's central argument about AI in practice
15	Pearsh	2026-10-29	Claude	Draft the commentary templates	Four paragraphs: cost, schedule, diagnosis by package, forecast	**Modify**	Changed the index band wording from "behind plan" to "below plan". A cost index is never "behind" — it was schedule idiom applied to a cost measure, which would have read as sloppy to a practitioner
16	Akshat	2026-10-29	Claude	Design the manual benchmark workbook so that it is genuinely independent	Proposed typed Excel formulas acting on raw inputs, with no value carried across from Python	**Accept**	Checked the independence claim carefully: the two sides share *inputs* but not *calculations*. Stated explicitly in the workbook's Read Me and in validation report §6
17	Akshat	2026-10-29	Claude	The benchmark workbook's cells are empty when read back — diagnose	openpyxl writes formulas but no cached results; a reader trusting the cache sees nothing. Fix: set `wb.calculation.fullCalcOnLoad = True`	**Accept**	Verified by round-tripping through LibreOffice and reading 25 computed values per period
18	Akshat	2026-10-29	Claude	How should the Excel benchmark be compared against the Python engine?	Recommended **executing** the workbook via LibreOffice headless rather than re-implementing the formulas in Python, citing the prior-art argument that a Python re-implementation validates only our own understanding	**Accept**	This reshaped the validation strategy and is the single most consequential suggestion in the project. Had we re-implemented, a misread formula would have been misread identically on both sides and the validation would have passed
19	Akshat	2026-10-29	Claude	Run the full benchmark comparison	50 of 50 comparisons agree, to ₹0.01 on currency and 1e−9 on indices, across both periods	**Accept**	Output inspected measure by measure rather than taken on the summary line. The identity checks (TCPI/EAC1 = CPI) hold on the Excel figures too
20	Neel	2026-10-29	Claude	Apply the chart-default fixes documented in the prior-art repository	Four overrides: `x_axis.auto = False`, `invertIfNegative = False`, `tickLblPos = "low"`, and `NA()` instead of `""` beyond the data date	**Accept**	Each is invisible to XML inspection — an element you never wrote has no XML to read back — so added a test asserting each override is set, and opened the workbook to confirm the charts are legible
21	Neel	2026-10-29	Claude	Build the live-formula Excel export	Workbook with Dashboard/Inputs/Baseline/EVM/Variance sheets, EV computed by `SUMPRODUCT` over budgets and percentages	**Accept**	Verified by evaluating the export through LibreOffice: every figure matches the engine to the paisa. Kept strictly separate from the benchmark — a workbook the tool generated cannot validate the tool

#	Member	Date	Tool	Purpose / prompt summary	Output summary	Decision	Reason and verification
22	Neel	2026-10-29	Claude	Lay out the Streamlit dashboard	Proposed: verdict and plain-language summary first, then indices, then diagnosis, then forecast	**Accept**	Ordering defended on the grounds that an EAC without knowing what drives it gives a manager nothing to act on. Checked by opening the running app and reading it as a user would
23	Neel	2026-10-29	Claude	**Bug found by looking at the running app: the executive summary is invisible**	Custom CSS set light panel backgrounds without setting a text colour; under a dark browser preference Streamlit supplied white text, rendering white-on-white	**Accept**	**No unit test would have caught this.** Every number was correct and the tool was unusable. Fixed with explicit colours and a pinned light theme. Recorded as discrepancy 5.3
24	Neel	2026-10-29	Claude	Set charting conventions for the dashboard	Curves stop at the data date; forecast dashed and branching; index axes pinned not auto-scaled	**Accept**	Each convention has a stated reason in `app/charts.py`. The pinned axis matters most: an auto-scaled index axis makes the same CPI look catastrophic or negligible depending on the rest of the series
25	Neel	2026-10-29	Claude	Review the S-curve forecast tail	Axis label read `+7.4p`, which is cryptic	**Modify**	Relabelled to `forecast finish ~P13.4` and moved the EAC value into the legend entry
26	Dependar	2026-10-29	Claude	Review the case study for realism before submission	Flagged that the baseline ran Apr-2025 to Mar-2026, putting the "current" data date a year in the past at submission	**Accept**	Shifted to Apr-2026 – Mar-2027 (data date Sep-2026). Numerically identical; benchmark rebuilt and revalidated at 50/50, screenshots retaken
27	Pearsh	2026-10-29	Claude	Draft the method note, user manual and validation report	Full drafts	**Modify**	Substantially restructured. The AI's first draft of the validation report led with the 50/50 result; moved "what this validation does **not** establish" from an appendix to a prominent section, because it is the more important finding
28	Pearsh	2026-10-29	Claude	Write the limitations sections	Comprehensive list across manual, charter and workbook	**Accept**	Verified each limitation is genuinely true of this implementation rather than generic hedging. The percent-complete limitation is repeated in four places deliberately
29	Pearsh	2026-10-30	Claude	Draft the learnings and reflection report against the six sections the brief specifies	Full draft, including the AI capability assessment and the ethics and limitations sections	**Modify**	Rewrote §2 substantially. The draft's assessment of where AI was weakest was generic ('struggles with ambiguity'); replaced with the two specific failures we actually observed — it could not detect the invisible-text bug because that is not a property of code, and it proposed a validation strategy that would have validated our own misunderstanding until the prior-art argument was put to it
30	Pearsh	2026-10-30	Claude	Draft the presentation slide content, demo script and viva preparation	12 slides with timings, a demonstration checklist and ~30 anticipated questions with answers	**Modify**	Checked every figure quoted on a slide against the engine output rather than against the draft. Added the algebraic proof that TCPI against EAC1 reduces to CPI, because an examiner asking why two of our reported measures are identical is a question we should not have to derive on the spot

#	Member	Date	Tool	Purpose / prompt summary	Output summary	Decision	Reason and verification
31	Pearsh	2026-10-30	Claude	Draft scaffolds for the five individual reflections in §5 of the reflection report, from each member's role and the discrepancies attributable to it	Five first-person half-page entries against a common four-question structure	**Modify**	**The most important verification in this log.** An AI-drafted account of a person's own learning is not that person's reflection, and submitting one as such would be exactly the misrepresentation the brief's academic-integrity clause describes. The drafts were used as a structure and a prompt only; each member rewrote their own entry in their own words and confirmed its content before submission. The factual anchors in each — the EAC3 rounding error, the evenly-spread case study, the dark-theme bug, the two mis-constructed tests — are drawn from the discrepancy log and are matters of record
32	Neel	2026-11-05	Claude	Add a data-upload route to the dashboard so a reviewer can run the tool on their own project, plus a template download	A sidebar data-source selector, a four-file uploader writing to a temporary folder, and a zipped copy of the case study as the editable template	**Accept**	Tested by loading the four case-study files through the upload path and confirming the engine returned CPI 0.8715 at Period 6, identical to the folder path. The template ships the *worked* case study rather than blank files deliberately: a baseline that must reconcile to each package budget, and packages that must sum to BAC, is far easier to preserve by editing than to achieve from an empty form
33	Akshat	2026-11-05	Claude	Generate a second case study in the same four-file format, to test the upload route	A 15 MLD water treatment plant, 13 packages, BAC Rs 8.50 crore, deliberately under budget and behind programme	**Modify**	Accepted the data after checking it loads and that the engine returns CPI 1.041 and SPI 0.827 as designed. The run immediately exposed a defect in our own commentary: a project forecast to underrun was told it needed a **-5% improvement** in cost efficiency, because the closing sentence was generated unconditionally from TCPI over CPI. Fixed, logged as discrepancy 5.5, and pinned by a new regression test. Worth recording that the case study was built to exercise untested branches and found a defect on its first run
34	Neel	2026-11-05	Claude	Diagnose a reported empty Excel download and two sidebar controls that appeared to do nothing	Root cause of both, plus a fix for each	**Modify**	Verified before fixing rather than after. Reading the workbook with cached values only showed 29 of 46 Dashboard cells populated, all of them text labels; passing the file through LibreOffice headless raised it to 46 with the numbers present, which proves the file is valid and the viewer was not calculating. Rejected the suggestion to write values instead of formulas, which would have deleted the workbook's main feature; added a notice and a plain-number fallback instead. The two controls were tested in the running app and both work — the fault was that each drives a single tab and neither said so

#	Member	Date	Tool	Purpose / prompt summary	Output summary	Decision	Reason and verification
35	Akshat	2026-11-05	Claude	Make the Excel export bake in computed values by passing the saved workbook through LibreOffice headless, so it opens correctly in viewers that do not calculate	A guarded post-save recalculation step, falling back silently where LibreOffice is absent	**Reject**	It worked: readable cells on the Dashboard went from 29 to 47, the formulas survived, and the =NA() gaps that stop the earned and actual curves at the data date were preserved. Rejected because two existing tests failed on the result. The round trip resets `x_axis.auto` to True and drops explicit series titles — the exact chart settings we had added to stop Excel promoting the category axis to a date axis. It would have traded numbers that look blank in a previewer for charts that render wrongly in Excel, which is worse. Kept the notice and the plain-number fallback instead. The tests we wrote weeks earlier are what caught it

2. Notable entries expanded

The table above satisfies the format. These five entries are where the critical judgement actually happened.

2.1 Entry 12 — the AI was right and our hand calculation was wrong

Our hand calculation of EAC method 3 gave ₹17,98,32,249. The engine gave ₹17,98,32,093, a difference of ₹156.

The instinct on finding a discrepancy between a hand check and a tool is that the tool is wrong — that is, after all, the point of a hand check. Instead of assuming, the quantity was recomputed in 40-digit decimal arithmetic. The engine matched full precision exactly. We had rounded the composite index CPI × SPI to six decimal places partway through, which is what one naturally does on paper and which looks harmless; because EAC3 divides a ₹10.5 crore remainder by that index, it is not.

Three things came out of this that would not have come out of simply accepting the engine's number:

1. A rule for the rest of the project: **rounding belongs in presentation only, never in the calculation chain.** The benchmark workbook now rounds through cell number formats alone.
2. A regression test that fails if anyone reintroduces mid-chain rounding.
3. A genuine piece of subject learning. ₹156 on an ₹18 crore forecast is immaterial, but the same habit applied to CPI early in a project — when EV is small and the divisor volatile — moves the forecast by lakhs.

The AI's contribution was the investigation method, not the answer. Asking "which of these two is right, and prove it independently" is the move that mattered, and it is available to anyone.

2.2 Entry 13 — resisting the easy fix

Two validation tests failed. The fastest route to a green test suite would have been to adjust the engine until they passed.

Diagnosis showed the engine was correct in both cases: an earlier validation rule was firing before the rule under test, because the test data was broken in two ways at once. "Fixing" the engine would have meant **deleting a correct validation rule to satisfy a badly written test** — and the rule in question rejects progress reported against a summary WBS branch, which is a real way for earned value to be double-counted.

This is the failure mode the brief warns about, in miniature. The AI produced both the mis-constructed tests and the correct diagnosis. What determined the outcome was asking *why* the test failed before changing anything.

2.3 Entry 14 — rejecting an AI feature on principle

The brief asks for "automatic plain-language monthly progress commentary", and the obvious implementation is to pass the indices to a language model. We asked whether we should, and the argument that came back was against it. We adopted that argument:

- **Reproducibility.** The same inputs must produce the same sentence every month. "The model phrased it differently this time" is not a defence in front of a client or a lender.
- **It cannot invent.** A language model given a table of indices will readily produce a fluent paragraph containing a number that is not in the table. In `evm/commentary.py` there is no path by which a figure reaches the text without having been computed, and `test_every_figure_quoted_is_a_computed_figure` asserts it.
- **Auditability.** Every sentence traces to a threshold written down and justified in one file, where a reader who disagrees can find it.

The tool is therefore *less* AI-powered than it could be, and the reduction is the point. The place AI earned in this project is upstream — drafting those templates, checking formula logic, writing code, catching our arithmetic slip — not generating the numbers a decision rests on.

2.4 Entry 23 — what no test catches

The dashboard's executive summary rendered white-on-white under a dark browser theme. Every figure was correct. All 128 tests at the time passed. The tool was unusable.

It was found by opening the running application and looking at it.

The lesson is narrow and worth stating because it cuts against the rest of this project's emphasis: validation establishes that the arithmetic is right. It says nothing about whether the thing works. "Validated" and "usable" are different claims, and only one of them can be automated.

2.5 Entry 31 — the limit of what AI could write for us

The five individual reflections in §5 of the reflection report are the one deliverable where AI assistance is constrained by what the deliverable *is*, rather than by whether the output would be any good.

Claude can write a fluent, specific, plausible account of what a person learned, because it has the discrepancy log and the role allocation and can infer what the Validation Engineer must have found difficult about a rounding error. The output reads well. It is also, as a reflection, worth nothing: a reflection is a first-person claim about a particular person's understanding, and nobody but that person can make it. Submitting an AI-written account of our learning as our learning would be the precise misrepresentation the brief's academic-integrity clause describes, and it would collapse the first time an examiner asked one of us about our own paragraph.

We used the drafts as a structure and a prompt — the four questions, and a reminder of which recorded discrepancy each of us owned — and each member then wrote their own entry. The factual anchors are matters of record from the discrepancy log; the understanding described is each member's own.

This is the clearest case in the project of AI output being useful at one remove: valuable as scaffolding, inadmissible as product.

3. Where AI helped, and where it did not

Genuinely accelerated

- Boilerplate: data loading, pivoting, Excel layout, Streamlit scaffolding. Hours of typing, little judgement.
- Recall of standard formulas, correctly, including the less-used ones (TCPI against EAC, the weighted EAC4).
- Operational knowledge that would otherwise cost a day of debugging — `fullCalcOnLoad`, the Excel chart defaults, the LibreOffice headless invocation.
- Adversarial review of its own earlier output (entry 6).
- Arithmetic verification at a precision we would not have attempted by hand (entry 12).

Did not help, or needed correcting

- **Judgement about what matters.** Its first validation report led with "50 of 50 agree". The more important finding is that both implementations share inputs, so neither can detect a wrong percent-complete figure. We moved that to a prominent section; it did not propose doing so.
- **Domain idiom.** "Behind plan" for a cost index (entry 15) is the kind of error that signals to a practitioner that the author has not worked on a site.
- **Case study realism.** Its first draft spread variance evenly across packages, which would have made the central feature pointless.
- **Anything visual.** Entry 23.

The pattern. AI was strongest where the answer is determinate and checkable — a formula, a library call, a precision question. It was weakest where the question is "what matters here?", which is most of engineering judgement and all of professional responsibility.

---

4. Accountability

Every number in every deliverable has been verified against an independent manual benchmark, and the comparison is reproducible by anyone with the repository and LibreOffice.

Where AI output was accepted, it was accepted after checking. Where it was wrong, that is recorded here and in the validation report. Where we were wrong and it was right, that is recorded too.

The team is accountable for all of it. "The AI generated it" is not a defence, and the main reason to keep a log in this much detail is that writing down *why* each output was accepted forces the check to actually happen.

CE725 CONSTRUCTION ECONOMICS AND FINANCE

Topic 7 — Earned Value Management Dashboard

Learnings and Reflection Report

Team 40

Sajjad Nakhwa	23B0702	Project Lead and Finance Modeller
Dependar Arya	23B0331	Data Lead
Neel Shah	23B4244	AI and Tool Developer
Akshat Karkar	23B4229	Validation Engineer
Pearsh Gumasta	23B4232	Documentation and Reflection Lead

Semester course project · November 2026

Team 40 — Sajjad Nakhwa (23B0702) · Dependar Arya (23B0331) · Neel Shah (23B4244) · Akshat Karkar (23B4229) · Pearsh Gumasta (23B4232)

Contents

1. Subject learnings
2. AI capability assessment
3. Validation insights
4. Practical relevance
5. Individual reflections
6. Ethics and limitations

1. Subject learnings

1.1 The third number

Before this project our mental model of project cost control was the one most people carry: compare what you have spent against what you planned to spend, and the difference is the problem.

The case study broke that model in a way a lecture had not. At the data date the project had spent ₹3.98 crore against a planned ₹4.02 crore. By the conventional reading it is marginally **under** budget and the report writes itself. It is in fact both significantly overspending and significantly behind programme, and the apparent health is a coincidence of two errors pointing in opposite directions — less work done than planned, each unit of it costing more than planned.

The insight is small to state and large in consequence. **Two numbers can only produce one comparison.** Budget versus spend conflates "am I doing the work" with "is the work costing what it should", and there is no way to separate them without a third measurement. Earned value — the budgeted cost of the work actually performed — is that third number, and everything else in EVM is bookkeeping on top of it.

What surprised us is how cheap the third number is. It requires no new cost system. It requires someone to state, per work package, what fraction is complete. The barrier to EVM in Indian construction practice is not computational; it is that nobody is made accountable for that fraction.

1.2 Indices are not the point; the forecast is

We began thinking CPI and SPI were the output. They are not. They are intermediate quantities whose value is almost entirely that they feed a forecast.

A CPI of 0.872 means nothing to a client. "At current performance this project finishes at ₹16.06 crore against a budget of ₹14.00 crore" means a great deal, because it is a number someone must now find or write off. The index matters because it supports that sentence.

This changed the tool. An early version led with a grid of indices. The final version leads with a plain-language paragraph and puts the forecast in the third tile row, because that is the order in which the questions are actually asked.

1.3 The four EAC methods, and why there is no right one

We expected to learn "the" formula for estimate at completion. There isn't one, and understanding why was among the more valuable things in the project.

The four standard methods differ in exactly one assumption: what you believe about the cost performance of the work that remains.

Method	Belief	Case study result
EAC1 = BAC / CPI	Performance to date is typical and continues	₹16.06 Cr
EAC2 = AC + (BAC − EV)	The variance was a one-off; the rest runs at budget	₹14.51 Cr
EAC3 = AC + (BAC − EV)/(CPI × SPI)	Cost *and* schedule pressure continue	₹17.98 Cr
EAC4 = weighted	Cost dominates; schedule partly weighted	₹16.09 Cr

The spread is **₹3.47 crore on a ₹14 crore project** — a quarter of the contract value. That spread is not a defect in the method; it is the method being honest. Narrowing it requires a judgement nobody can make arithmetically: was the rock excavation a closed event, or a symptom of a site that was never properly investigated?

Two mathematical observations clarified the family for us. First, **TCPI against EAC1 equals CPI exactly** — a forecast built on "performance continues" demands, by construction, no change in performance. It is a tautology, and recognising it is what shows EAC1 is a *projection*, not a *plan*. Second, **VAC on EAC2 equals the cost variance already incurred** — if the remainder runs at budget, the final overrun is precisely the overrun to date. Both identities now sit in the test suite, where they also serve as a check that the whole chain is intact.

Our conclusion, which we would defend in practice: **report the range.** A single EAC implies a precision the method does not possess.

1.4 TCPI is the sharpest instrument in the set

The most useful single number we encountered was the one we knew least about beforehand.

TCPI = (BAC − EV) / (BAC − AC) is the cost efficiency the remaining work must achieve to land on budget. On the case study it is **1.051**, against a CPI the project has sustained at **0.872** for six consecutive periods. Finishing on budget requires a **20.6% improvement** on performance never once achieved.

What makes it powerful is that it converts an argument into an arithmetic statement. A recovery plan claiming the budget can still be met is now claiming something specific and falsifiable. In a room where a contractor is presenting a recovery plan and a client is deciding whether to believe it, TCPI is the question that ends the discussion.

We also learned when it stops meaning anything: once AC reaches BAC the denominator is zero or negative, and there is no efficiency at which the remaining work can be done within a budget that is already gone. The engine returns NaN rather than a number, which we think is the honest behaviour.

1.5 Earned Schedule, and a defect we would not have noticed

The most substantial piece of subject learning came from a question we asked almost in passing: why does SPI behave oddly near the end of a project?

The answer is structural. SPI = EV / PV, and at completion EV must equal BAC and PV must equal BAC. **Therefore SPI returns to 1.00 on every project ever completed, however late.** Schedule variance returns to zero. The indices improve through the final third and the improvement is an artefact of the arithmetic. A planner watching SPI receives their most reassuring reading at the moment the project is latest.

This is not a subtlety; it is a hole in a measure that appears in every textbook and most contracts. Earned Schedule fixes it by asking when on the baseline the current earned value was scheduled to have been earned, which yields a **time**:

```
ES     = 5.357 months of baseline value earned
AT     = 6 months elapsed
SV(t)  = −0.64 months  ≈ 19 days behind
SPI(t) = 0.893
```

"About nineteen days behind programme" is actionable. "SPI is 0.863" is not. We implemented both and display both, with the gap explained, rather than quietly preferring one — partly because practitioners know SPI and would distrust a tool that omitted it, and partly because the disagreement between them is itself informative.

1.6 The empirical regularity that changes how you read a trend

The literature reports that CPI tends to stabilise early in a project — often by the 20% complete point — and rarely recovers thereafter.

We built the case study to exhibit this, and seeing it plotted taught us more than reading it. Across six periods CPI sits between 0.86 and 0.88 while SPI swings from 0.88 down to 0.74 and back to 0.86. Schedule performance is volatile; cost performance is not.

If that regularity holds, a flat below-1.00 CPI is strong evidence against any recovery plan that assumes cost performance will improve. Combined with TCPI, it gives a reviewer a defensible basis for scepticism rather than a hunch.

We are careful to report this as a **finding from the literature that our synthetic case was built to illustrate**, not as something this project has independently verified. One constructed dataset demonstrates nothing empirically. That distinction is exactly the kind we were least careful about before this project.

1.7 Aggregation destroys the diagnosis

The project's main functional decision was to carry the work breakdown structure through every calculation rather than aggregating to a single project curve.

A project CPI of 0.872 says something is wrong. The breakdown says: **₹38 lakh of the ₹51.20 lakh adverse variance sits in three substructure packages, all complete, so that portion is final and no recovery plan touches it**; only ₹7.7 lakh, in the frame packages still running, is live.

Those are different conversations. The first invites a generic cost-control exercise. The second says: stop trying to recover the substructure, protect the frame, and tell the client ₹38 lakh is gone.

The prior-art tool we studied rolls each project to a single percentage curve and lists the absence of a WBS breakdown as a known limitation. We now understand why that limitation is the significant one.

---

2. AI capability assessment

The project was built with Claude (Anthropic) throughout. Full log in `docs/05-ai-usage-log.md`. This section assesses capability honestly, with specific examples in both directions.

2.1 Where AI was genuinely strong

Operational knowledge that would have cost days. Three examples. That openpyxl writes formulas but no cached results, so a workbook read back appears empty unless `fullCalcOnLoad` is set. That LibreOffice can be driven headlessly to evaluate a workbook, and needs a dedicated user profile or it exits silently when another instance is open. That Excel treats an absent `invertIfNegative` as *true* and fills negative bars white, so an all-negative cost variance chart renders as hollow outlines.

Each of these is the kind of thing that costs an afternoon of confused debugging and appears in no textbook.

Formula recall, accurate. Every EVM formula it produced was correct when checked against the standard definitions, including the less-used ones — TCPI against EAC rather than BAC, the weighted EAC4, the Earned Schedule interpolation. We checked all of them. None was wrong.

Arithmetic verification beyond our patience. When our hand calculation of EAC3 disagreed with the engine by ₹156, it recomputed the quantity in 40-digit decimal arithmetic and demonstrated that the engine matched full precision and our hand figure had rounded mid-chain. We would have accepted the ₹156 as rounding noise and moved on, and would have missed a rule worth adopting.

Adversarial review of its own output. Asked to review the progress carry-forward routine it had written an hour earlier, it identified a nested loop indexing a pivot table by a column that might not exist. The defect had not yet produced a wrong answer, because the case study has no gaps in reporting — but real project data does.

Arguing against its own use. Asked whether the monthly commentary should call a language model, it argued against, on grounds we adopted (§2.4 below). We note this is also the kind of answer a model might give because it reads as appropriately cautious, which is why we evaluated the argument on its merits rather than on its source.

2.2 Where AI was wrong or needed correcting

It does not know what matters. Its first draft of the validation report opened with "50 of 50 comparisons agree" and relegated the limitations to an appendix. But the more important finding is that both implementations read the same inputs, so **neither can detect a wrong percent-complete figure**. We restructured the report to lead with the limitation. It did not propose this, and would not have.

This is the central limitation. It optimises for the question asked. It does not ask whether the question is the right one.

Domain idiom. The commentary described a cost index as "behind plan". A cost index is never behind; schedule is behind, cost is above or below. Small, and exactly the kind of thing that signals to a practising engineer that the author has not worked on a site.

Case study realism. Its first draft spread cost variance evenly across all work packages. Arithmetically fine, and it would have made the project's central feature — locating variance — pointless, because everything would have been equally to blame. It optimised for "produce a case study with a variance" rather than "produce a case study that demonstrates why this tool exists".

Anything visual. The dashboard's executive summary rendered white-on-white under a dark browser theme. Every number correct, all 128 tests passing, tool unusable. Found by opening it and looking.

It generated the mis-constructed tests too. Two validation tests were written in a way that triggered an earlier rule than the one under test. It also diagnosed them correctly when asked. The pattern is consistent: good at producing plausible work, good at critiquing work when directed to, not reliable at noticing unprompted that its own output is wrong.

2.3 A specific instance of wrong-in-a-plausible-way

The most instructive error was ours, caught by it — but it illustrates the risk precisely.

Had the EAC3 discrepancy been ₹156 in the *other* direction — had the engine been wrong and our hand calculation right — would we have caught it? Only because we had a hand calculation at all. A team that had accepted the engine's figure, formatted it to two decimal places and put it in a report would have had a wrong forecast presented with six-figure precision and complete confidence.

That is the characteristic risk of AI-generated analysis: not that it is obviously wrong, but that it is wrong in a way that looks exactly like being right. EVM output is particularly exposed — three ratios near 1.00, no intuition a reader can apply. A CPI of 0.94 from corrupt data looks exactly like a CPI of 0.94 from good data, and it will be taken to a client meeting.

2.4 The decision not to use AI in the product

The brief asks for "automatic plain-language monthly progress commentary". The obvious implementation passes the indices to a language model. We decided against it, and the decision is the project's main argument about AI in professional practice.

Three reasons:

1. **Reproducibility.** The same inputs must produce the same sentence every month. A cost report is defended line by line, and "the model phrased it differently this time" is not a defence.
2. **It cannot invent.** A language model handed a table of indices will readily produce a fluent paragraph containing a number that is not in the table. In `evm/commentary.py` no figure can reach the text without having been computed, and a test asserts it.
3. **Auditability.** Every sentence traces to a threshold written down and justified in one file. A reader who disagrees with "within 5% is noise" can find it and argue.

So the tool is *less* AI-powered than it could be, and that is the point. **The right division is that AI belongs upstream of the number, not between the number and the reader.** It drafted the templates, checked the formulas, wrote the code and caught our arithmetic slip. It does not generate the sentence a client acts on.

2.5 What it changed about how we work

The most durable change is a habit: **ask the question that could prove it wrong, and ask it in a different system.**

The validation strategy came from a question of this kind — "how would we know if we had misunderstood the formula?" — and the answer, that a Python re-implementation would embed the same misunderstanding, is why the benchmark is executed in a spreadsheet engine instead. That habit is not about AI at all. It is ordinary engineering discipline, which AI makes more necessary because it makes plausible output so much cheaper to produce.

---

3. Validation insights

3.1 What the comparison showed

Fifty comparisons across two reporting periods, agreeing to ₹0.01 on currency and 1×10^{-9} on dimensionless indices. Two independently written implementations — Python and typed Excel formulas — of the same definitions.

3.2 The insight that reshaped the strategy

Our first plan was a second Python implementation to compare against.

The argument against it, from the prior-art repository's documentation, is that *a self-check that re-implements the intended logic in Python only validates your own understanding — it will agree with your bug.* If we had misread a formula, the re-implementation would embed the same misreading, agree perfectly, and the validation would pass while both were wrong. The test would measure our consistency, not our correctness.

So the benchmark lives in a different execution environment with different semantics, and it is **executed, not re-implemented**. LibreOffice loads the workbook, recalculates every cell, and we read what a real spreadsheet engine computed.

The general principle, which we expect to carry into professional work: **a check is only worth what is independent about it.** Two implementations by the same person, in the same language, from the same understanding, are one implementation run twice.

3.3 What the discrepancies taught us

Four surfaced. Details in `docs/04-validation-report.md` §5.

The ₹156 EAC3 difference taught us that rounding is not cosmetic. Rounding CPI × SPI to six decimal places mid-chain moved an ₹18 crore forecast by tens of rupees. Immaterial here; the same habit applied to CPI early in a project moves it by lakhs. Rounding is for presentation only.

Two mis-constructed tests taught us the discipline of asking *why* a test fails before changing anything. Both failures were in the tests. The quick fix would have deleted a correct validation rule — the one rejecting progress booked against a summary WBS branch, which is a real way for earned value to be double-counted.

The dark-theme bug taught us that "validated" and "works" are different claims. Every number correct, every test passing, screen unreadable. Only one of those claims can be automated.

The carry-forward routine, caught in review rather than by a test, taught us that a defect invisible on clean data is exactly the defect that surfaces on real data. The case study has no gaps in reporting. Real projects do.

3.4 The most important thing validation did not do

This is the finding we would lead with if we could only report one.

The benchmark validates the calculation. It cannot validate the input.

Both implementations read the same budgets, the same percent-complete figures, the same costs. That is deliberate — a benchmark against different data would compare nothing. But it means that if a work package is reported at 60% complete when it is really 40%:

- the Python engine computes a wrong earned value,
- the Excel benchmark computes **the same** wrong earned value,
- all 50 comparisons agree perfectly,
- every index on the dashboard is confidently, precisely wrong,
- and nothing in the software can detect it.

Earned value is percent complete multiplied by budget. Percent complete is a judgement made at site, often by a person with an interest in the answer. A contractor's engineer assessing their own progress ahead of an interim payment application is not a neutral observer.

The practical consequence is that the governance around how percent complete is assessed, by whom, and with what independent check, matters more than the software. It is also the part most likely to be neglected, because it is the part that cannot be bought, installed or demonstrated in a fifteen-minute presentation.

We find this genuinely uncomfortable, because we have spent the project building the part that does not matter most.

---

4. Practical relevance

4.1 Would anyone use it?

A contractor's planning engineer: probably yes, with changes. The work-package breakdown and the exported live-formula workbook are directly useful. The blockers are integration — the tool wants three clean CSVs, and a real planning engineer has a Primavera P6 export and a Tally cost ledger that do not share a coding structure. Reconciling those is most of the real work and the tool does none of it.

A client's project management consultant: yes, with the least change. This is the best fit. A PMC reviewing a contractor's monthly report wants exactly what the tool provides: an independent recomputation of the indices, a view of where variance sits, and TCPI as a test of whether the recovery plan is arithmetically possible. A PMC already receives progress data in summary form, so the integration problem is smaller.

A lender's independent engineer: yes, for the forecast range. The question is whether sanctioned debt will cover the project. The four-EAC spread and TCPI answer it directly, and the honest presentation of a range rather than a point estimate is closer to how a lender thinks.

A site engineer: no. Wrong altitude. They need to know which pour is late this week, not that CPI is 0.872.

4.2 What would have to change for real use

In rough order of how much each would block adoption:

1. **Schedule integration.** The tool reads a time-phased budget, not a network. It can say the project is 19 days behind in aggregate. It cannot say whether the slipped package is on the critical path, and therefore cannot say whether the completion date has moved. For a contractor arguing an extension of time this is the gap that matters most, and closing it means ingesting a P6 or MS Project file.
2. **Re-baselining.** One baseline per project. Real projects are re-baselined after major variations, and a tool that cannot represent that will be abandoned at the first one.
3. **Variations and claims.** Scope changes are assumed already incorporated. In practice a variation's cost is known long before its budget is formally approved, and projects spend months in a state where work is being done against a budget that does not yet include it.
4. **Cost ledger integration.** Manual CSV preparation each month will not survive contact with a busy site.
5. **Retention, advance and cash flow.** EVM measures cost and progress, not liquidity. A project can be green on every index here and still fail because it cannot pay subcontractors this month. In Indian construction this is the more common failure mode.
6. **Multi-project roll-up.** A PMC or a lender monitors a portfolio.
7. **Audit trail.** Who reported 22% complete on the columns, when, and on what basis. Given §3.4, this may matter more than anything else on this list.

4.3 The organisational obstacle

Everything above is solvable with engineering time. The real obstacle is not technical.

EVM requires someone to state, monthly and in writing, what fraction of each work package is complete — and then be held to it when the figure is compared against cost. That is a transfer of accountability, and it is resisted for exactly that reason. It is more comfortable for everyone if progress stays qualitative ("substructure is nearly done") because qualitative progress cannot be shown to have been wrong.

A tool makes the arithmetic free. It does not make the accountability free, and the accountability was always the expensive part. We think this explains why EVM adoption in Indian construction is lower than the obvious benefit would predict, and it is not something better software will fix.

4.4 Testing with a practitioner

The brief offers bonus credit for testing with a practising engineer. We have not been able to do this within the timeframe.

What we would ask, were the opportunity available, and we would structure it to seek disconfirmation rather than approval:

1. Show the dashboard without explanation and ask what they conclude. (Tests whether the plain-language summary works.)
2. Ask how they currently assess percent complete, and who checks it. (Tests §3.4 against practice.)
3. Show the TCPI of 1.051 against CPI of 0.872 and ask whether they would accept a recovery plan claiming the budget is still achievable.
4. Ask what in the monthly commentary they would delete before sending it to a client.
5. Ask what would stop them using this on Monday.

We expect question 2 to produce the most useful answer and question 5 the most deflating one.

---

5. Individual reflections

Each entry below is written by the member named, in their own words, and covers the work that member owned.

5.1 Sajjad Nakhwa (23B0702) — Project Lead and Finance Modeller

I owned the method: which measures we would report, which forecasting formulas we would use, and the conventions for signs, rounding and undefined values that everyone else then had to build against. I also set the order of the work and settled the arguments — and I got one of those badly wrong.

I came into this expecting to learn *the* way to forecast what a project will finally cost. There isn't one. There are four standard methods that differ in a single assumption each, and on our case study they span ₹3.47 crore, a quarter of the budget. My first instinct was that this was the method failing, and I specified that the tool should choose the most appropriate one and display a single number. Neel and Akshat both pushed back, and they were right. That feature would have been the most confidently misleading thing we built. A single EAC does not remove the uncertainty — it removes the reader's awareness of it, which is worse, because they then decide believing they know something they do not.

Being argued out of that in week two is the most useful thing that happened to me here. Looking back, I reached for the single number because a range felt like an admission that we did not know. It was. We did not know. Pretending otherwise would have transferred our uncertainty onto whoever read the screen, without telling them.

The second mistake was quieter. I left the arithmetic conventions unwritten because rounding rules and sign conventions are boring and I wanted to get to the interesting part. We then lost most of a day to a ₹156 disagreement between Akshat's hand calculation and the engine, which existed only because nobody had said where rounding was permitted. The rule we settled on — rounding is presentation only, never intermediate — would have taken five minutes to write on day one.

What I will carry into practice is TCPI read against CPI as the first question to ask of any recovery plan. Our project has sustained 0.872 for six months and needs 1.051 on everything remaining. That comparison takes ten seconds and turns "we will pull it back in the second half" from a reassurance into a claim with a number attached. I find myself thinking about the site team who would be asked to

deliver that 20% improvement, and about the client who absorbs ₹38 lakh either way. Both are better served by being told in month six than at handover.

5.2 Dependar Arya (23B0331) — Data Lead

I built the case study: the work breakdown, the budget for each of the 19 packages, the baseline window each one sits in, the phasing within those windows, and the progress and cost figures for all six reported periods. I also wrote the statement of what our numbers are and are not, which mattered more than I expected it to.

My first version spread the overrun evenly across all 19 packages. It produced a realistic-looking project CPI and I was pleased with it. Akshat pointed out that it made our main feature pointless: if everything is equally to blame, telling a manager where the variance is tells him nothing the project total had not already. So I rebuilt it around a story — hard rock in excavation, finishing two months late and 45% over, cascading into the footings and the foundation RCC, pushing the frame a month late. Now 74% of the variance sits in three packages that are already finished, which is a completely different management problem from the same rupees sitting in live work.

What I had not understood before is how quietly a baseline fails when it does not tie out. The time-phased budget has to sum to each package's budget, and the packages have to sum to BAC. If they do not, PV is wrong by the error, every schedule index is silently scaled by it, and nothing visible breaks — the S-curve still looks plausible and SPI still reports to three decimals. I had assumed a model would announce its own errors. It does not. The reconciliation check that now refuses to load a project whose baseline does not balance exists because of that.

I would work the other way round next time. I chose numbers first and reverse-engineered a story to fit them, and spent longer doing that than it would have taken to write "hard rock in the excavation" first and derive everything from it.

What I carry is a discomfort I did not have before. A number on a screen acquires authority it has not earned unless something on the same screen is actively denying it. Ours is synthetic — every figure assumed — and we say so in the charter, the method note, the manual, the workbook and on the dashboard itself. That felt excessive while I was writing it. It does not now. I would far rather over-label an illustration than have someone quote our ₹14 crore as though we had priced a real building.

5.3 Neel Shah (23B4244) — AI and Tool Developer

I built the thing itself: the Python engine, the Streamlit dashboard, the charts, the live-formula Excel export, and the repository.

The failure I keep returning to is the dark-theme bug. I styled the executive summary panel with a light background and never set a text colour. On a browser in dark mode Streamlit supplied white text, and the single most important paragraph in the tool — the plain-language verdict a non-specialist reads first, and may be the only thing they read — rendered white on white. All 151 tests passed. Every number was correct. The tool was unusable. I found it by accident, opening the app to take screenshots for the manual.

That changed how I think about testing. A test suite is evidence about arithmetic and says nothing whatever about whether a person can read the output. For most of the build my only feedback was pytest output: fast, precise, and blind to everything I most needed to know. I should have opened the running app every single day.

The other lesson was about defaults nobody documents. `openpyxl` writes formulas into a workbook but no cached results, so any reader that trusts the cache sees empty cells — one flag fixes it, and nothing warns you. Excel fills negative bars white unless told otherwise, so a variance chart renders with the adverse packages invisible. Neither is a bug in my code. Both produce a file that is correct and useless. Writing code that produces a correct file turns out to be a different problem from writing code that produces a file which *opens* correctly, and only the second one matters to the person receiving it.

What I will carry into practice is that when generated code leans on a library's default, I check what that default actually is rather than assuming silence means sensible. And I keep picturing the person at the other end — a planning engineer opening our workbook on a Monday with a meeting at eleven. They do not care that the arithmetic is right if the chart is blank.

5.4 Akshat Karkar (23B4229) — Validation Engineer

I owned the check: the independent benchmark workbook, the hand calculations for Periods 5 and 6, the 151-test suite, and the discrepancy log.

My first plan was to write a second implementation of the EVM formulas in Python and compare the two. It feels rigorous. It is close to worthless, and understanding why was the most useful thing I learned all semester. Both implementations would have been written by the same people from the same reading of the same formulas. A formula I had misunderstood would have been misunderstood identically on both sides: the comparison passes, I report agreement, and both numbers are wrong. So the benchmark became typed Excel formulas acting on the raw inputs, *executed* by LibreOffice rather than re-implemented — a different formula language, evaluated by a different engine. That one decision is the difference between a validation and a reassurance, and I would not have found it if I had not been made to write down what exactly my check was independent of.

Then I got my own work wrong, twice, and both times usefully. My hand calculation of EAC3 disagreed with the engine by ₹156, and I assumed the engine was wrong because the engine was the thing under test and I was the control. It was not wrong. I had rounded CPI × SPI to six decimals partway through the chain, as one does on paper, and the error propagated through the division. The control is not automatically correct merely because it is the control. That one stung, and it should have.

I also wrote two tests that failed, both asserting that a validation rule would reject a bad project when an earlier rule caught it first. The fast route to a green suite was to loosen the engine until the tests passed. That would have deleted a correct rule — the one that catches earned value being double counted — and the project would have looked healthier while being worse.

What I will carry is to ask *why* a test fails before changing anything at all. Two of our six discrepancies were defects in the check rather than in the thing being checked, and in both cases the quick fix would have made the output greener and the tool less trustworthy. That asymmetry — that the wrong fix looks like progress — is what I want to remember.

5.5 Pearsh Gumasta (23B4232) — Documentation and Reflection Lead

I owned everything written: the user manual, the AI usage log, this report, and the presentation material.

What I had not understood is how much a document's structure argues, independently of its contents. Our first validation report opened with "50 of 50 comparisons agree" and put in an appendix the fact that neither implementation can detect a wrong percent-complete figure at source. Every sentence in it

was true, and the document said the tool was verified. Moving that limitation into a prominent numbered section, with nothing else changed, turns the same facts into a document saying the arithmetic is verified and the inputs are not. A reader who skims the first page and stops takes away the opposite impression from identical evidence, because of an ordering decision I had made without thinking about it.

I also nearly let the AI usage log become a list of uniformly positive entries. It would have been accurate — most AI output was in fact accepted — and it would have been evidence that nobody was checking anything. The entries worth reading are the handful where something went wrong: where the AI was wrong, where it was right and we were wrong, and where we rejected output that worked perfectly well, on principle. I restructured the log around those, and said plainly why a uniformly positive log should not be believed.

I would write the limitations section first next time. It is the section that gets written last, when everyone is tired, the tool is working, and nobody wants to dwell on what it cannot do — and it is the first section a practising engineer turns to. Writing it first would also have improved the tool, because several of those limitations were fixable in week one and not in week four.

What I carry is to repeat the most important caveat rather than state it once. The percent-complete limitation appears in five places in our documents, deliberately, and I expected that to feel laboured. It does not, because a polished interface works continuously to make the reader forget it, and one mention in a method note is no match for a dashboard showing a confident verdict in red and green.

Mostly I keep thinking about who is at the end of all this. Earned value rests on a percent complete judged at site, often by someone under pressure to report a higher number, sometimes by someone whose payment depends on it. We validated the arithmetic to nine decimal places. We never once validated that. If this tool were used for real, the honest thing to tell whoever relies on it is that we checked the sums, not the truth of what went into them.

6. Ethics and limitations

6.1 Data reliability

Every figure in the case study is **assumed**. Budgets are order-of-magnitude plausible for an institutional building built to CPWD DSR-style item rates; they are not taken from any tender, estimate or invoice, and no claim is made that they are accurate unit rates.

This is stated in the charter, the method note, the data generator's docstring, the dashboard's data tab and the exported workbook. The reason for the repetition is that a polished dashboard confers unearned authority. A number rendered in a large clean typeface next to a green tick reads as a fact, and ours are not facts.

The decision to use synthetic rather than real data was deliberate (method note §2.2) and it has a cost: we cannot claim the tool has been tested against the messiness of real data — missing periods, retrospective corrections, packages whose scope changed, costs booked to the wrong code. Stating what our validation does not cover is part of the validation.

6.2 Over-reliance on AI

The project was built with substantial AI assistance and we think the result is better for it. The risk is specific and worth naming precisely.

AI makes plausible output cheap. Writing a wrong EVM engine used to take as long as writing a right one; both now take minutes. The distinguishing effort has shifted almost entirely to verification, and verification is the part that is tedious, unglamorous and easy to skip when the output looks finished.

Our structural defence was to make verification independent: a manual benchmark in a different execution environment, hand-calculated expected values written before the engine ran, algebraic identities that hold only if the whole chain is intact. None of that is novel engineering. It is ordinary discipline, which AI makes *more* necessary rather than less.

The honest limit of our defence: it catches arithmetic and logic errors. It does not catch **errors of framing** — building the wrong thing correctly. Nothing in our test suite would have told us if EVM were the wrong method for this problem, and an AI optimising for the question asked will not raise its hand about the question.

6.3 Accountability for decisions made on AI-generated numbers

If a client cancels a contract on the strength of a forecast this tool produced, and the forecast was wrong, who is responsible?

Not the tool, and not the AI that helped write it. The engineer who signed the report. That is unchanged by the method of production, and it is worth stating because the fluency of AI-generated analysis invites a diffusion of responsibility that the law and professional practice do not recognise.

Three practical positions we adopted:

Traceability. Every number in every deliverable can be traced to an input and a formula. The benchmark workbook shows the arithmetic in visible cells. The commentary cannot contain a figure that was not computed. Someone who must defend the output can see how it was produced.

Refusing to compute rather than computing from bad data. The tool rejects inconsistent input rather than proceeding. This is deliberate and sometimes inconvenient. EVM output is three ratios near 1.00 with no sanity check a reader can apply; a CPI of 0.94 from corrupt data looks exactly like a CPI of 0.94 from good data. Refusing to produce a number is safer than producing one that cannot be challenged.

Disclosure. The AI usage log is a deliverable, not an appendix. It records where AI was wrong, where we were wrong, and where its output was rejected on principle.

6.4 The ethics of a convincing interface

The uncomfortable conclusion of this project is that we have spent most of our effort on the part that matters least.

The arithmetic is validated to nine decimal places. The arithmetic was never the risk. **The risk is the percent-complete figure typed into a spreadsheet by someone who will be paid more if it is higher**, and we have built an instrument that renders that figure with six-figure precision, a traffic-light verdict and a chart.

Every design choice that makes the dashboard more persuasive makes this worse. A tool that looked crude would be trusted in proportion to its reliability. Ours looks authoritative, and authority is exactly what it has not earned, because it cannot see its own inputs.

Our response was to put the limitation where it cannot be missed — on the dashboard, in the manual, in the charter, in the exported workbook, and leading the validation report's conclusions. That is a partial answer and we know it. A user in a hurry will read the number and not the caveat.

If there is one thing we would want a reader of this report to take away, it is that the quality of an EVM system is the quality of its percent-complete measurement, and everything else — including all of the work in this repository — is arithmetic on top of a judgement made by a person standing on a site.

6.5 Scope limitations, collected

Limitation	Consequence
Percent complete is an unverifiable site judgement	**All indices inherit its error. The governing limitation.**
No critical path or network	Cannot say whether a slip moves the completion date
No re-baselining	Unusable after a major variation
No retention, advance or cash flow	A project can be green here and still run out of money
Variations assumed already in the budget	A real gap between work done and budget approved
Equal-length periods assumed	Unequal periods distort the trend charts
Single project	No portfolio roll-up
Synthetic data only	Untested against real ledger messiness
Nominal rupees, one price base	No escalation
Chart legibility not machine-checkable	Needs a human to open the file

CE725 CONSTRUCTION ECONOMICS AND FINANCE

Topic 7 — Earned Value Management Dashboard

Presentation Script and Viva Preparation

Team 40

Sajjad Nakhwa	23B0702	Project Lead and Finance Modeller
Dependar Arya	23B0331	Data Lead
Neel Shah	23B4244	AI and Tool Developer
Akshat Karkar	23B4229	Validation Engineer
Pearsh Gumasta	23B4232	Documentation and Reflection Lead

Semester course project · November 2026

The brief states that **any team member may be asked to explain any formula or result**. Section 3 of this document is the preparation for that, and is the part worth rehearsing.

---

0. The built deck

The slides below are built into `docs/presentation/CE725_Team40_EVM_Dashboard.pptx` — 12 slides, 16:9, with the *Say* paragraphs carried through as speaker notes.

The deck is **generated**, not hand-built, by `docs/presentation/build_deck.js`:

```
NODE_PATH=$(npm root -g) node docs/presentation/build_deck.js
```

Every figure quoted on a slide is listed in one `FIG` block at the top of that script, so a number on a slide can be checked against the engine rather than taken on trust — the same reason the rest of the project is built the way it is. Slides 6 and 7 carry the dashboard screenshots from `docs/img/`, so they work unchanged if the live demonstration cannot be run.

---

1. Slide content and timing

Total 15 minutes. Timings are cumulative. The live demonstration is the centre of it — slides 6 to 9 are driven from the running dashboard, not from bullet points.

Slide 1 — Title (0:00–0:30)

Earned Value Management Dashboard Construction Economics and Finance · CE725 · Topic 7 Team 40 · Sajjad Nakhwa · Dependar Arya · Neel Shah · Akshat Karkar · Pearsh Gumasta

Say: "We built a tool that tells a project manager whether a project is in trouble, where the trouble is, and what it will cost. The part we want to spend most of the time on is how we know the numbers are right — and what that checking still cannot tell us."

Slide 2 — The problem, in one number (0:30–2:00)

> **A project has spent ₹3.98 crore against a plan of ₹4.02 crore. Is it healthy?**

Pause. Let the room answer "yes".

> It has completed ₹3.47 crore of work. It is **14.7% over budget** and **19 days behind programme**.

Say: "Two numbers give you one comparison. Budget against spend conflates 'am I doing the work' with 'is the work costing what it should'. Here the two errors point in opposite directions and cancel, and the report writes itself as healthy. You cannot separate them without a third measurement."

The third number: earned value — the budgeted cost of work actually performed.

Slide 3 — The method (2:00–3:30)

PV	Budgeted cost of work **scheduled**	What should have been done?
EV	Budgeted cost of work **performed**	What was done?
AC	Cost of doing it	What did it cost?

Everything else is a comparison of two of these three:

- EV vs AC → cost: **CV = EV − AC**, **CPI = EV / AC**
- EV vs PV → schedule: **SV = EV − PV**, **SPI = EV / PV**

Say: "That is the whole method. The rest is bookkeeping on top of it."

Slide 4 — The case study (3:30–4:30)

G+4 RCC academic block · BAC ₹14.00 crore · 12 months · 19 work packages · data date end of month 6

The story the numbers tell: - Hard rock in excavation — 45% over on that package, two months late - Cascaded into footings and foundation RCC - Frame started a month late, running an 18-day slab cycle against 12

Say, explicitly: "Synthetic data. Budgets are order-of-magnitude plausible for a building of this size, but they are assumed, not from any tender or invoice. We'll come back to why that choice matters."

Slide 5 — What we built (4:30–5:00)

Four pieces, one repository:

1. **Python engine** — all EVM measures, work-package level
2. **Streamlit dashboard** — the tool
3. **Live-formula Excel export** — for people who work in Excel
4. **Independent manual benchmark** — how we know it is right

→ **Switch to the live dashboard.**

Slide 6 — DEMO: the position (5:00–7:00)

Show the top of the dashboard.

Point at, in order: 1. The verdict bar — **BEHIND SCHEDULE AND OVERSPENT** 2. The plain-language summary — read it aloud; note it quotes no index without the consequence beside it 3. **CPI 0.872** — "every rupee spent returns 87 paise of budgeted work" 4. **TCPI 1.051** — "the remaining work must run at 1.051 to finish on budget. We have sustained 0.872 for six months. That is a 20.6% improvement on performance never once achieved. The budget is gone; what remains is deciding who absorbs it."

If you have time for only one sentence in the whole demonstration, it is the TCPI one.

Slide 7 — DEMO: where the variance is (7:00–9:00)

Open **Where the variance is**.

Say: "A project CPI of 0.872 says something is wrong. It does not say what to do. This does."

Point at: - Excavation, **CPI 0.689**, **100% complete** — "this variance is **final**. No recovery plan touches it." - Frame packages, **22% and 14% complete** — "these can still move." - "₹38 lakh of the ₹51.20 lakh is in completed substructure. Only ₹7.7 lakh is live. Stop trying to recover the substructure, protect the frame, and tell the client ₹38 lakh is gone."

This is the slide that justifies carrying the WBS through the whole calculation instead of rolling up to one project curve.

Slide 8 — DEMO: the schedule defect (9:00–10:30)

Open **Performance trend**.

Say: "SPI reads 0.863. There is a problem with that number, and it is structural.

SPI = EV / PV. At completion, EV must equal BAC and PV must equal BAC. **So SPI returns to 1.00 on every project ever completed, however late.** The index gets more reassuring as the project gets later.

Earned Schedule fixes it by asking *when* on the baseline this much value was scheduled to have been earned. That gives a time: **19 days behind**. A site manager can act on nineteen days. Nobody can act on 0.863."

Also point at **CPI flat at 0.86–0.88 across all six periods** while SPI swings to 0.74 and back: "cost performance stabilises early and rarely recovers. That is the strongest argument against an optimistic recovery plan."

Slide 9 — DEMO: the forecast (10:30–11:30)

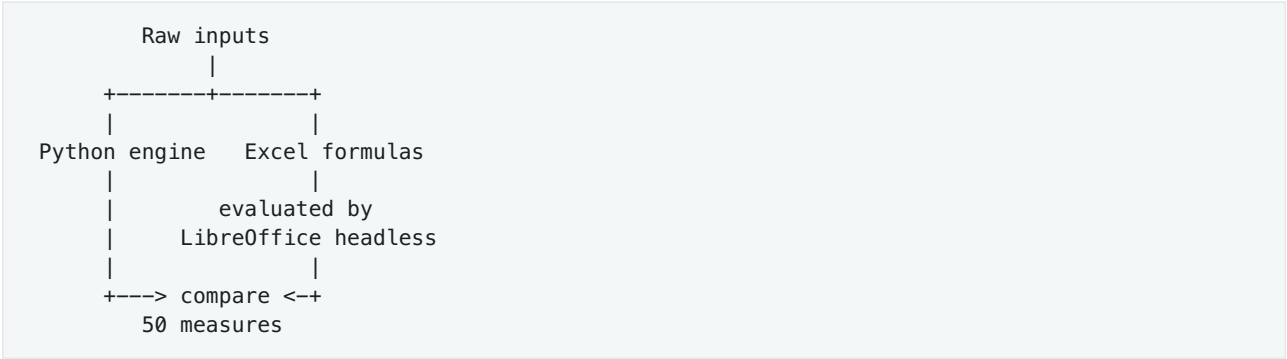
Open **Forecast**.

Four methods, ₹14.51 Cr to ₹17.98 Cr.

Say: "We deliberately do not pick one. They differ in a single assumption — what you believe about the remaining work. The ₹3.47 crore spread is the method being honest. Narrowing it needs a judgement nobody can make arithmetically: was the rock a closed event, or a symptom of a site that was never properly investigated?"

→ **Back to slides.**

Slide 10 — How we know it is right (11:30–13:00)



50 of 50 agree — ₹0.01 on currency, 1e−9 on indices, both periods. **151 automated tests.** Expected values hand-derived before the engine was written.

Say: "The critical decision was **not** to write a second Python implementation. A Python re-implementation validates only your own understanding — if you misread the formula, both sides misread it identically and the check passes. So the benchmark is in a different execution environment, and it is *executed*, not re-implemented."

Slide 11 — What we got wrong (13:00–14:00)

Six discrepancies, all recorded:

Found	Resolution
EAC3 off by **₹156**	**Our hand calc was wrong**, not the engine — rounded CPI×SPI mid-chain. Rule adopted: rounding is presentation only
Two validation tests failing	**The tests were wrong, not the engine.** Fixing the code would have deleted a correct validation rule
Executive summary **invisible** under dark theme	No test caught it. Every number right, tool unusable
Carry-forward routine fragile on gaps	Caught in review. Clean data hid it; real data would not

Say: "The second one matters most. The fast route to a green test suite was to change the engine until the tests passed, and that would have removed a rule that rejects earned value being double-counted."

Slide 12 — What validation cannot do (14:00–15:00)

The closing point. Do not rush it.

> Both implementations read **the same inputs**.
>
> If a package is reported **60% complete when it is really 40%**: the engine computes a wrong EV · the benchmark computes **the same** wrong EV · all 50 comparisons agree · every index is confidently wrong.

Earned value is percent complete × budget. Percent complete is a judgement made at site by someone who may be paid more if it is higher.

Say: "We validated the arithmetic to nine decimal places. The arithmetic was never the risk. And every design choice that makes this dashboard more persuasive makes the problem worse — it renders an unverifiable judgement with six-figure precision, a traffic light and a chart.

The quality of an EVM system is the quality of its percent-complete measurement. Everything else, including all of our work, is arithmetic on top of a judgement made by a person standing on a site."

---

2. Demonstration checklist

Before presenting:

- [ ] `.venv/bin/streamlit run app/streamlit_app.py` — started and warm
- [ ] Browser at 100% zoom, window maximised
- [ ] Reporting period slider at **6**
- [ ] Variance breakdown set to **Sub-level**
- [ ] A pre-built `output/EVM_Report.xlsx` open in a second window, in case the live export is slow
- [ ] `.venv/bin/python -m pytest tests/ -q` run once — be ready to show **151 passed** if asked
- [ ] Repository open in a browser tab

If the app fails to start, present from the screenshots in `docs/img/` and the exported workbook. Do not debug live.

---

3. Viva preparation

Any member may be asked anything. These are the questions we expect.

On the formulas

Q. Derive CPI and explain what 0.872 means physically. CPI = EV / AC = ₹3,47,30,000 / ₹3,98,50,000 = 0.872. Of every rupee spent, 87.2 paise bought work that was budgeted at that value. The other 12.8 paise bought nothing that appears in the budget.

Q. Why can SPI not be trusted near the end of a project? SPI = EV / PV. At completion EV = BAC and PV = BAC by definition, so SPI → 1.00 regardless of how late the project finished. The index improves as the project gets later. Earned Schedule avoids this by measuring in time.

Q. Compute Earned Schedule for period 6. Cumulative PV is ₹3,16,63,556 at P5 and ₹4,02,48,444 at P6. EV is ₹3,47,30,000, which lies between them. ES = 5 + (3,47,30,000 − 3,16,63,556) / (4,02,48,444 − 3,16,63,556) = 5 + 30,66,444 / 85,84,889 = **5.357 months**. SV(t) = 5.357 − 6 = −0.643 months ≈ 19 days behind.

Q. Why are there four EAC formulas? They differ in one assumption: the cost performance of the remaining work. EAC1 assumes it continues as now. EAC2 assumes the variance was a one-off. EAC3

assumes schedule pressure also continues, which normally costs money because acceleration does. EAC4 weights cost and schedule.

Q. Show that TCPI against EAC1 equals CPI. TCPI = (BAC − EV)/(EAC − AC). Substituting EAC = BAC/CPI = BAC·AC/EV: denominator = BAC·AC/EV − AC = AC(BAC − EV)/EV. So TCPI = (BAC − EV)·EV / (AC(BAC − EV)) = EV/AC = **CPI**. *Meaning:* a forecast built on "performance continues" demands, by construction, no change in performance.

Q. When is TCPI undefined? When AC ≥ BAC. The denominator (BAC − AC) is zero or negative: there is no budget left to do the remaining work with, at any efficiency. The tool returns NaN rather than a number.

Q. Why is CV negative but you call it 14.7%? CV% = CV / EV, expressed against the work done rather than the budget. −₹51,20,000 / ₹3,47,30,000 = −14.7%. The work completed cost 14.7% more than it was budgeted to cost.

On the data

Q. Where did these numbers come from? They are synthetic, generated by `data/generate_case_study.py`, and we say so in every document. Budgets are order-of-magnitude plausible for an institutional building at CPWD DSR-style rates, but they are assumed. Real work-package cost data is commercially sensitive and would have required a confidentiality agreement we could not obtain in four weeks.

Q. Isn't synthetic data a weakness? Yes, and we state it. We cannot claim the tool has been tested against missing periods, retrospective corrections or miscoded costs. What synthetic data bought is a case that exercises every branch of the method — a complete over-budget package whose variance is final, a part-complete one that can still move, a cascading slip.

Q. How is the baseline phased? Each package has a start, a finish and a curve shape. S-shaped packages use smoothstep, $3t^2 - 2t^3$, which has zero slope at both ends so a package ramps up and tails off. The project S-curve is the aggregate of 19 staggered packages — it emerges from the phasing rather than being imposed.

On the tool

Q. Why Streamlit and not Excel? Both. Streamlit for interrogation — the period slider, the WBS drill-down. Excel for distribution, because a site QS works in Excel and a report they cannot re-run is one they will retype. The export is live formulas, not pasted values.

Q. Why is the commentary not generated by a language model? Three reasons. Reproducibility: the same inputs must give the same sentence every month. Non-invention: a model given a table of indices will write a fluent paragraph containing a number that is not in the table. Auditability: every threshold is written down in one file and can be argued with. The tool is deliberately less AI-powered than it could be.

Q. What happens if I feed it bad data? It refuses to load rather than computing an index. EVM output is three ratios near 1.00 with no sanity check a reader can apply, so a CPI of 0.94 from corrupt data looks exactly like one from good data. Refusing is safer.

On validation

Q. How do you know the tool is right? Two independent implementations — Python and typed Excel formulas — agree on 50 measures across two periods, to ₹0.01 and 1e−9. The Excel side is evaluated by LibreOffice, not re-implemented in Python.

Q. Why not just write a second Python version? Because it would validate our understanding against itself. If we misread a formula, the second implementation embeds the same misreading and the check passes while both are wrong.

Q. What did validation find? Four things. The most instructive: EAC3 differed by ₹156 and **our hand calculation was wrong**, not the engine — we had rounded CPI × SPI mid-chain. Also two mis-constructed tests where the engine was right, and a CSS bug that made the executive summary invisible and that no test could have caught.

Q. What does validation NOT prove? That the inputs are right. Both implementations read the same percent-complete figures. A package reported at 60% when it is really 40% produces the same wrong EV on both sides, all 50 comparisons agree, and every index is confidently wrong. Only a measurement at site catches that.

On practice

Q. Would a contractor actually use this? A client's PMC most readily — they already receive summarised progress and want an independent recomputation. A contractor's planner would need schedule integration first: the tool reads a time-phased budget, not a network, so it cannot say whether a slip is on the critical path.

Q. What is the biggest barrier to EVM adoption? Not computation. EVM requires someone to state monthly, in writing, what fraction of each package is complete, and then be held to it when that figure meets the cost data. That is a transfer of accountability, and it is resisted for that reason. Better software does not fix it.

On AI use

Q. How much of this did AI write? Most of the code, and first drafts of the documents. All of it was checked: expected values hand-derived before the engine ran, an independent benchmark, and a 35-entry usage log recording where AI was wrong, where we were wrong, and where its output was rejected on principle.

Q. Give an example of AI being wrong. Its first case study spread the variance evenly across packages, which would have made the tool's central feature pointless. Its first validation report led with "50 of 50 agree" and buried the limitation that matters more. And it described a cost index as "behind plan" — schedule idiom applied to cost.

Q. Give an example of AI being right when you were wrong. The ₹156 EAC3 discrepancy. We assumed the engine was wrong. It recomputed in 40-digit decimal arithmetic and showed our hand figure had rounded mid-chain.

Q. Your charter calls this an "AI-assisted EVM dashboard", but you say the commentary does not use a language model. Where is the AI? In how it was built, not in what it runs. AI wrote most of the code and the first drafts of the documents, and that is logged in full. The delivered tool is deterministic by choice — no model is called at runtime, because a cost report must give the same sentence from the

same inputs every month and must not contain a figure that was never computed. Deciding *not* to put a language model in the output path, and being able to say why, is the substantive finding of the project rather than a shortfall against the charter.

Q. Did AI write your individual reflections? It drafted a scaffold — the four questions and a reminder of which recorded discrepancy each of us owned — and each of us then wrote our own entry. That is logged as entry 31. A reflection is a first-person claim about one person's understanding, so an AI-written one is worth nothing however well it reads, and any of us can be asked about our own paragraph.

Q. Who is accountable if a decision on these numbers goes wrong? The engineer who signs the report. Not the tool, not the AI. That is why every number is traceable to an input and a formula, and why the tool refuses to compute from inconsistent data.

Earned Value Management Dashboard

Construction Economics and Finance · CE725 · Topic 7

Team 40 Sajjad Nakhwa · Dependar Arya · Neel Shah · Akshat Karkar · Pearsh Gumasta

₹14.00 Cr
project budget at completion

19
work packages tracked individually

50 / 50
independent benchmark comparisons agree

The problem, in one number

₹4.02 Cr
PLANNED VALUE
what should have been done

₹3.98 Cr
ACTUAL COST
what it has cost

₹3.47 Cr
EARNED VALUE
what was actually done

Spend is under plan by ₹4 lakh, so the monthly report writes itself as healthy.
The two errors point in opposite directions and cancel.

! **14.7% over budget**
every rupee spent returns 87 paise of budgeted work

! **19 days behind programme**
measured in time, not as a dimensionless index

Budget against spend is two numbers, and two numbers give one comparison. Separating ‘am I doing the work’ from ‘is the work costing what it should’ needs a third measurement: the budgeted cost of the work actually performed.

Three measurements, and comparisons between them

PV — **Budgeted cost of work scheduled**

What should have been done by now?

EV — **Budgeted cost of work performed**

What was actually done?

AC — **Actual cost of doing it**

What did it cost?

Everything else compares two of the three

EV against AC → cost

```
CV = EV − AC
CPI = EV ÷ AC
```

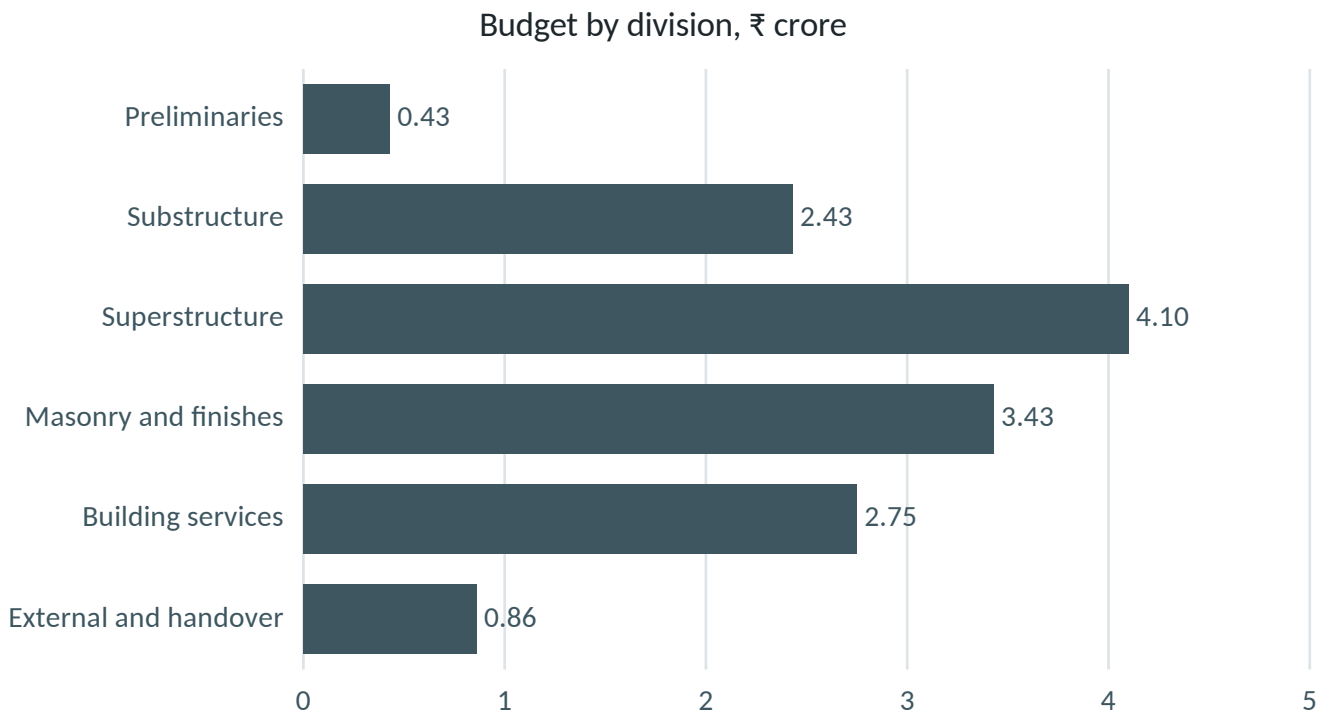
EV against PV → schedule

```
SV = EV − PV
SPI = EV ÷ PV
```

“That is the whole method. The rest is bookkeeping on top of it.”

The tool adds Earned Schedule, four forecasting methods and the same calculation at every work package.

The case study: a G+4 RCC academic block



BAC	₹14.00 crore
Baseline	12 months, Apr 2026 to Mar 2027
Structure	19 leaf work packages, 6 divisions
Data date	end of month 6, Sep 2026

How it went wrong

- Hard rock in excavation — 45% over on that package, two months late
- Cascaded into footings and foundation RCC
- Frame started a month late, running an 18-day slab cycle against 12

! **Synthetic data.** Budgets are order-of-magnitude plausible for an institutional building built to CPWD DSR-style item rates, but they are assumed — not drawn from any tender, estimate or invoice. Slide 12 is about why that choice matters more than it looks.

Four pieces, one repository

1

Python calculation engine

Every EVM measure, computed at work-package level rather than rolled up to a single project curve.

2

Streamlit dashboard

The tool. Verdict and plain-language summary first, then indices, diagnosis and forecast.

3

Live-formula Excel export

Every figure a real formula. Edit a budget or a percent complete and the whole workbook recalculates.

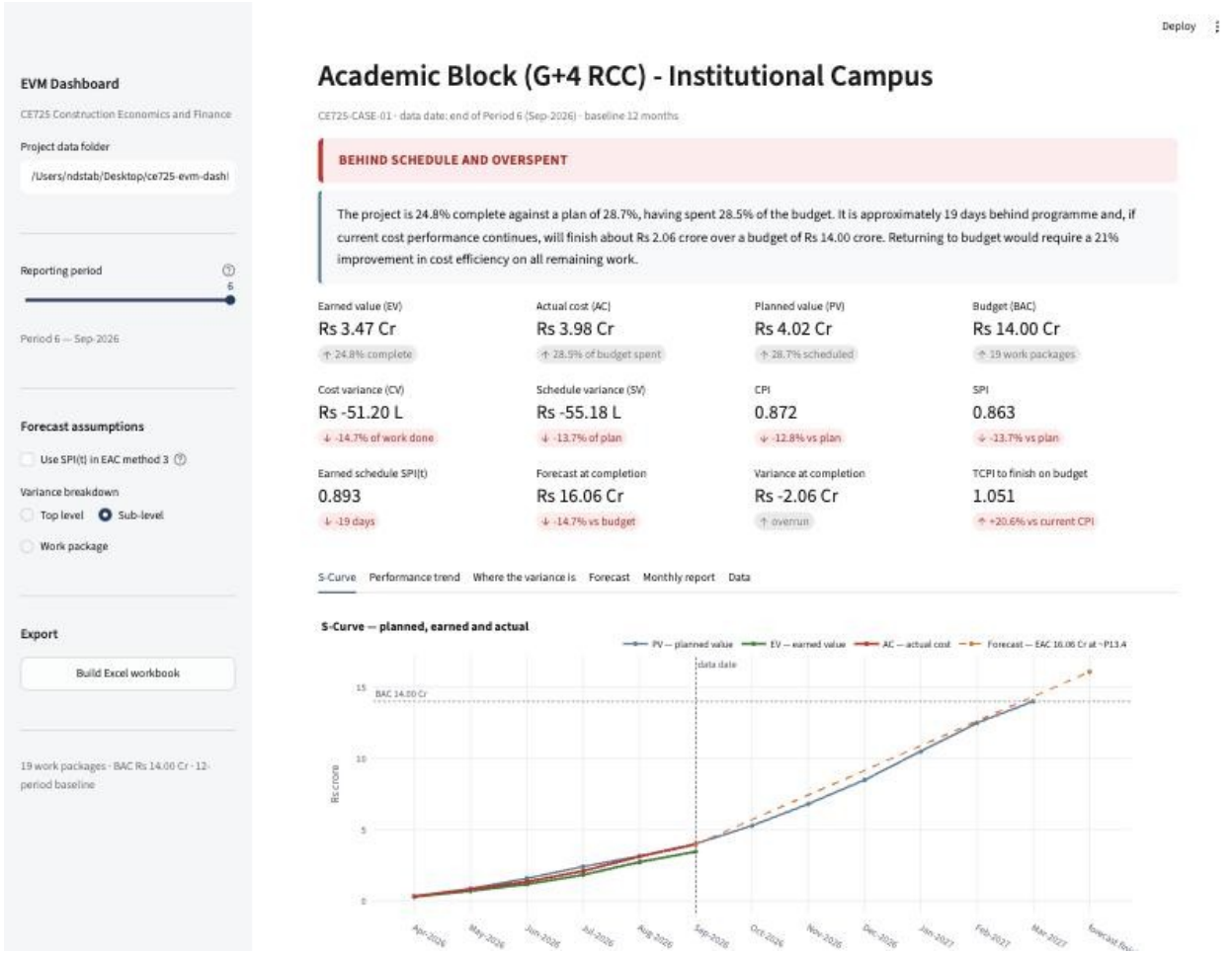
4

Independent manual benchmark

A separate Excel workbook, executed not re-implemented. This is how we know the numbers are right.

→ **Switch to the live dashboard** — slides 6 to 9 are driven from the running tool

Where the project stands at month six



CPI 0.872

every rupee spent returns 87 paise of budgeted work

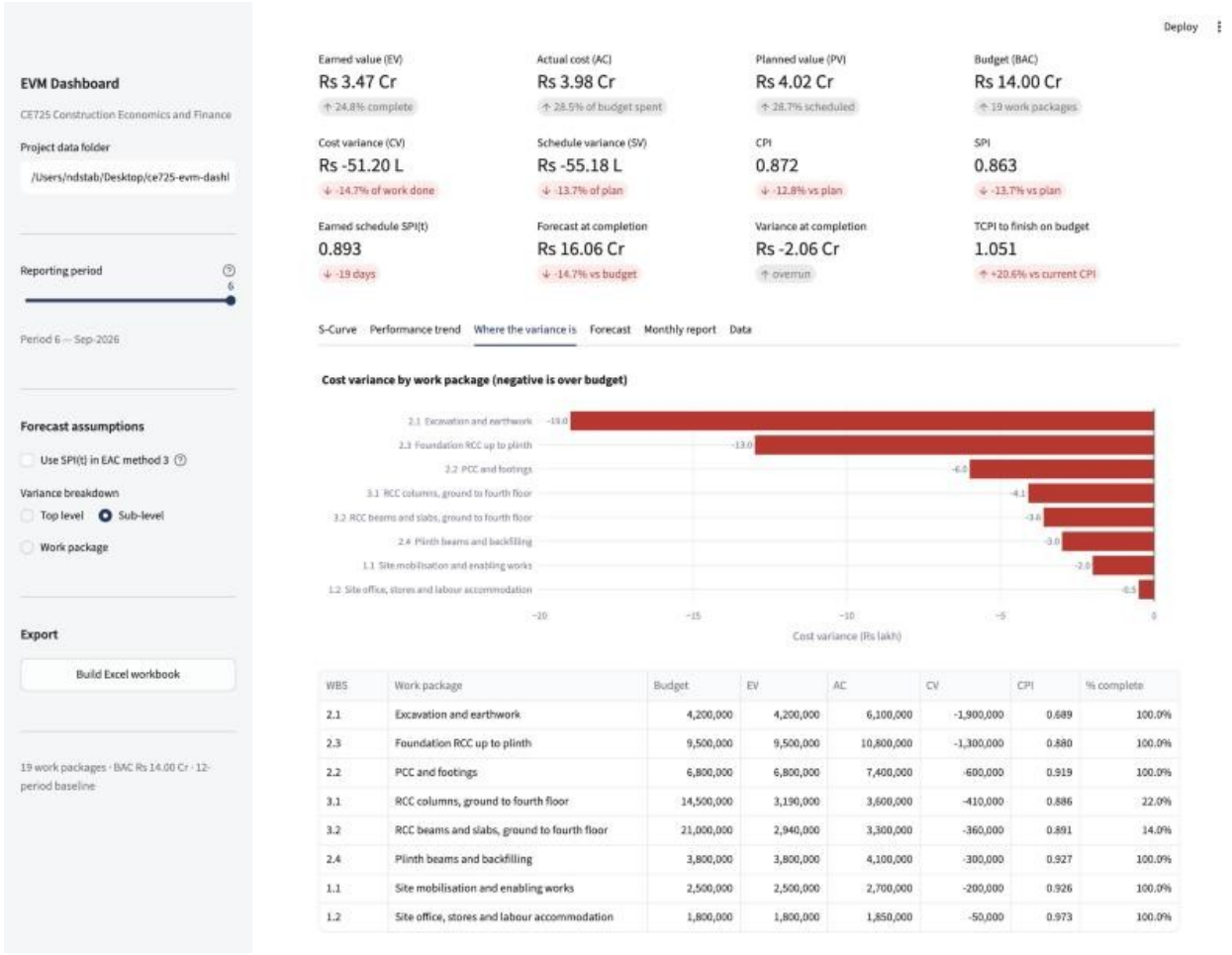
TCPI 1.051

efficiency the remaining work must now achieve

We have sustained 0.872 for six months. TCPI asks for a 20.6% improvement on a level of performance never once achieved.

The budget is gone. What remains is deciding who absorbs it.

Where the variance actually is



WBS	Work package	Budget	EV	AC	CV	CPI	% complete
2.1	Excavation and earthwork	4,200,000	4,200,000	6,100,000	-1,900,000	0.689	100.0%
2.3	Foundation RCC up to plinth	9,500,000	9,500,000	10,800,000	-1,300,000	0.880	100.0%
2.2	PCC and footings	6,800,000	6,800,000	7,400,000	-600,000	0.919	100.0%
3.1	RCC columns, ground to fourth floor	14,500,000	3,190,000	3,600,000	-410,000	0.886	22.0%
3.2	RCC beams and slabs, ground to fourth floor	21,000,000	2,940,000	3,300,000	-360,000	0.891	14.0%
2.4	Plinth beams and backfilling	3,800,000	3,800,000	4,100,000	-300,000	0.927	100.0%
1.1	Site mobilisation and enabling works	2,500,000	2,500,000	2,700,000	-200,000	0.926	100.0%
1.2	Site office, stores and labour accommodation	1,800,000	1,800,000	1,850,000	-50,000	0.973	100.0%

₹38 lakh

in three completed substructure packages — this variance is FINAL

₹7.7 lakh

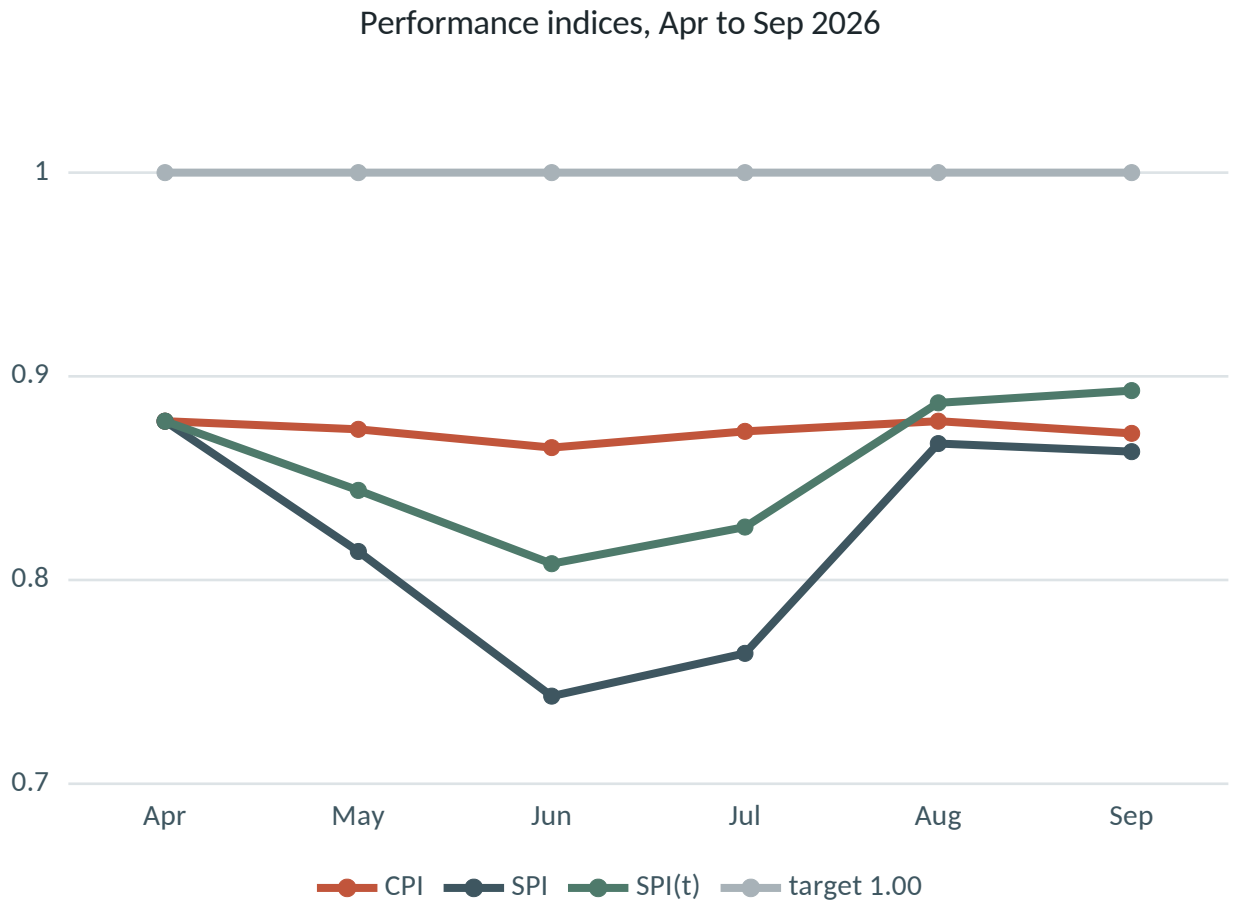
in the frame, 22% and 14% complete — this can still be managed

Excavation and earthwork: CPI 0.689, 100% complete.

No recovery plan touches it. Stop trying to recover the substructure, protect the frame, and tell the client ₹38 lakh is gone.

This is the slide that justifies carrying the WBS through the whole calculation instead of rolling up to one project curve.

SPI has a structural defect, and Earned Schedule fixes it



SPI = EV ÷ PV

At completion EV must equal BAC and PV must equal BAC.

So SPI returns to 1.00 on every project ever completed, however late.

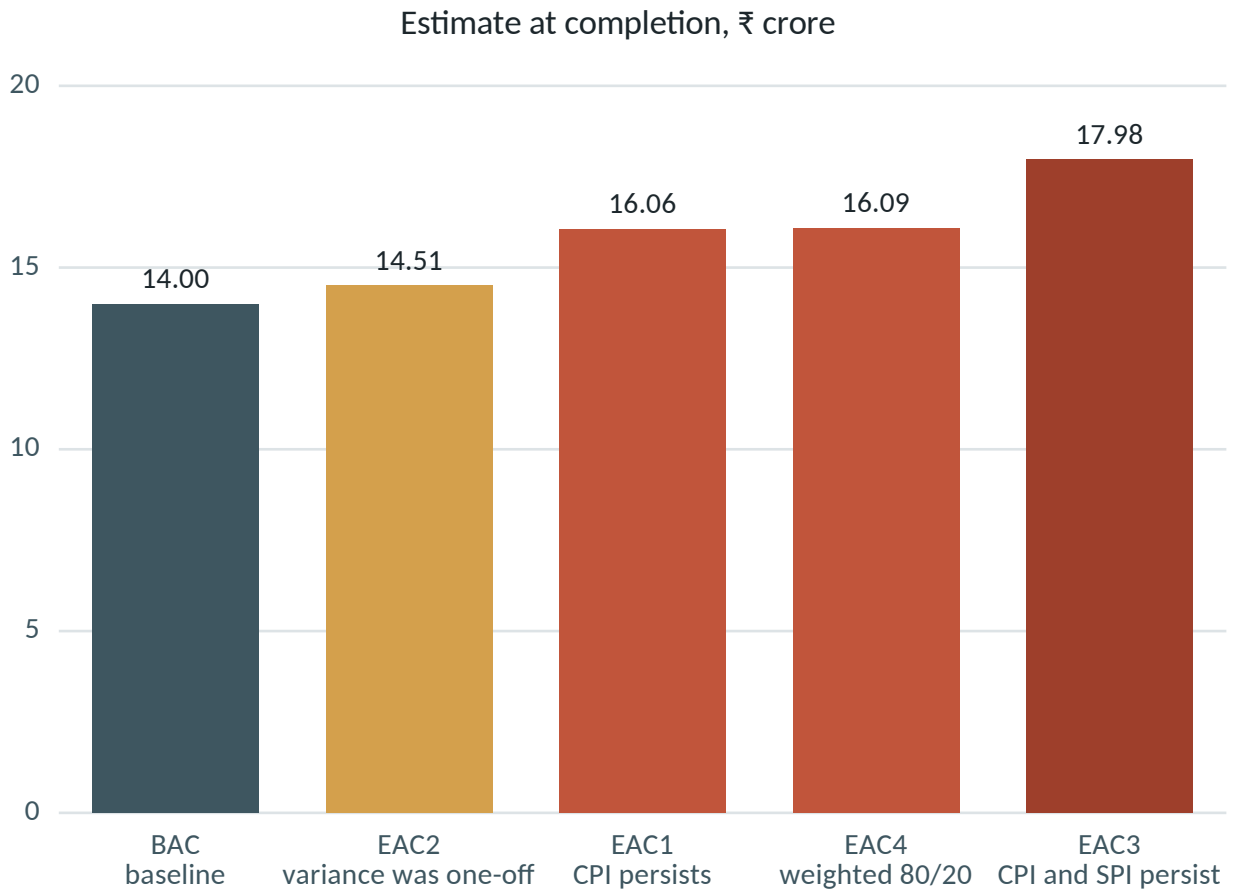
19 days behind

Earned Schedule, measured in time — SPI(t) 0.893

A site manager can act on nineteen days. Nobody can act on 0.863.

Note CPI flat at 0.86–0.88 across all six periods while SPI swings to 0.743 and back: cost performance stabilises early and rarely recovers.

The forecast is a range, and that is the honest answer



₹3.47 Cr

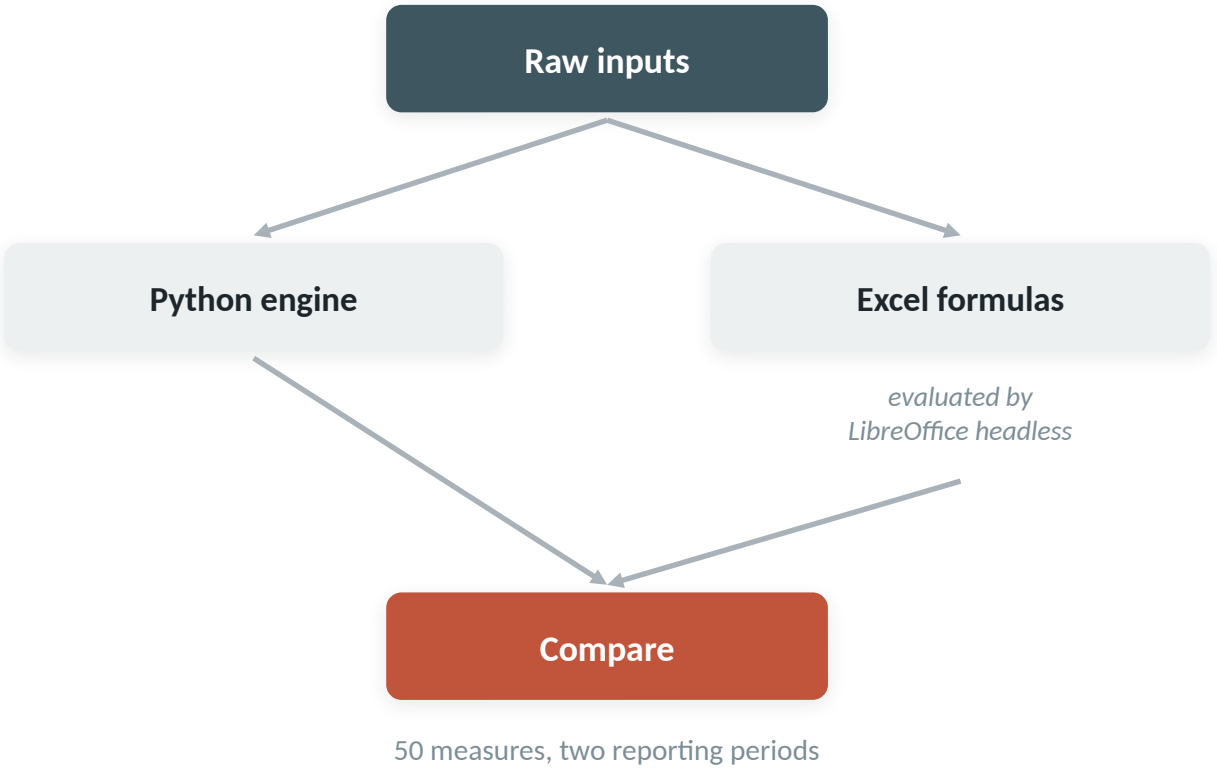
spread between the four standard methods

We deliberately do not pick one.

The four methods differ in a single assumption — what you believe about the remaining work. The spread is the method being honest, not failing.

Narrowing it needs a judgement nobody can make arithmetically: was the rock a closed event, or a symptom of a site that was never properly investigated?

How we know the numbers are right



50 / 50
comparisons agree — to ₹0.01 on currency, 1×10^{-9} on indices

150
automated tests — expected values hand-derived before the engine existed

The decision that mattered

Not to write a second Python implementation.

A Python re-implementation validates only your own understanding. Misread a formula and both sides misread it identically, the comparison passes, and both are wrong.

So the benchmark lives in a different execution environment, and it is executed — not re-implemented.

Four discrepancies, all recorded

What we found	How it resolved
EAC3 off by ₹156	**Our hand calculation was wrong, not the engine** — we rounded CPI × SPI mid-chain. Rule adopted: rounding is presentation only.
Two validation tests failing	**The tests were wrong, not the engine.** Fixing the code to make them pass would have deleted a rule that rejects earned value being double counted.
Executive summary invisible under a dark theme	**No test caught it.** Every number was correct and the tool was unusable. Found only by opening the running app.
Carry-forward routine fragile on reporting gaps	**Caught in review before it produced a wrong answer.** Clean data hid it; real data would not.

The second one matters most. The fast route to a green test suite was to change the engine until the tests passed — which would have made the project worse while making the output greener. Ask why a test fails before changing anything.

What none of this validation can do

Both implementations read the same inputs.

60% reported
40% actual

1. the engine computes a wrong EV
2. the benchmark computes the same wrong EV
3. all 50 comparisons agree
4. **every index is confidently, precisely wrong**

Earned value is percent complete × budget. Percent complete is a judgement made at site, often by someone who is paid more if it is higher. No spreadsheet and no test suite detects an error in it — only a measurement at site does.

We validated the arithmetic to nine decimal places. The arithmetic was never the risk.

And every design choice that makes this dashboard more persuasive makes the problem worse: it renders an unverifiable judgement with six-figure precision, a traffic light and a chart.

The quality of an EVM system is the quality of its percent-complete measurement.